A PROJECT REPORT ON

SalesPulse — Small Business Sales Analytics System

An Automated Point-of-Sale, Inventory Ledger, and Financial Analytics Platform

Submitted in partial fulfillment of the requirements for the award of degree of
BACHELOR OF TECHNOLOGY
in
COMPUTER ENGINEERING

Submitted By:
Janmesh Ravindra Rohida
(Roll No: SP-2026-042)

Under the Guidance of:
Prof. S. K. Sharma
Assistant Professor

Department of Computer Engineering, University Institute of Technology

Academic Year 2026–27

CERTIFICATE OF APPROVAL

This is to certify that the project entitled "SalesPulse — Small Business Sales Analytics System" is a bonafide work carried out by Janmesh Ravindra Rohida (Roll No: SP-2026-042) under my guidance and supervision in partial fulfillment of the requirements for the award of Bachelor of Technology in Computer Engineering at Department of Computer Engineering, University Institute of Technology during the academic year 2026–27.

The results embodied in this report have been verified and have not been submitted to any other University or Institution for the award of any degree or diploma.

Prof. S. K. Sharma
Internal Project Guide
Department of Computer Engineering

Head of Department
Department of Computer Engineering

CANDIDATE'S DECLARATION

I, Janmesh Ravindra Rohida, hereby declare that the work presented in this project report entitled "SalesPulse — Small Business Sales Analytics System" is an authentic record of my own work carried out under the supervision of Prof. S. K. Sharma.

I have not submitted the matter embodied in this report for the award of any other degree or diploma to any other university or institute. All ideas, algorithms, architectural patterns, and literature cited from other publications have been fully acknowledged and cited.

_____

Janmesh Ravindra Rohida
Roll No: SP-2026-042
Date: October 2026

ACKNOWLEDGEMENT

I express my deepest gratitude to my esteemed project guide, Prof. S. K. Sharma, for their continuous encouragement, valuable technical advice, and perceptive critique throughout the design and implementation of SalesPulse.

I am profoundly grateful to the Head of Department and the faculty members of Department of Computer Engineering, University Institute of Technology for providing state-of-the-art computational infrastructure, laboratories, and an inspiring academic environment.

Finally, I extend my heartfelt thanks to my family, friends, and peers whose moral support and constant motivation made this endeavor possible.

ABSTRACT

In traditional small retail commerce—including neighborhood kirana stores, stationery shops, bakeries, and mobile accessory outlets—transactional records are predominantly recorded in physical paper registers or unlinked spreadsheets. This manual paradigm creates operational bottlenecks: store owners cannot accurately ascertain real-time gross or net profit, identify high-turnover products, detect capital tied up in dead inventory, or anticipate customer demand.

To resolve these challenges, this project presents SalesPulse, a high-performance, web-based sales analytics, inventory tracking, and financial intelligence system developed using Python 3, Flask 3.x, MySQL 8, and modern web visualization technologies. SalesPulse models the fundamental retail financial identity: Net Profit = Gross Revenue − Cost of Goods Sold (COGS) − Operating Overheads. Unlike naive database systems that introduce accounting errors when product catalog prices change, SalesPulse implements historical snapshotting of unit cost and unit selling price on every transaction line item, preserving the mathematical integrity of historical financial statements.

SalesPulse features an interactive Point of Sale (POS) billing terminal with sub-second autocomplete, automated stock deduction with ACID transaction guarantees and row locking, low-stock and slow-moving inventory alerts, automated overhead recording, dynamic RFC-4180 CSV and vector PDF reporting, and an interactive 6-chart analytics dashboard powered by Chart.js. The system incorporates an autoregressive 7-day Simple Moving Average (SMA) forecasting model with weekday seasonality weighting. Deployed and validated through 44 automated test cases and Playwright browser verification across a seeded 90-day retail dataset, SalesPulse delivers enterprise-grade operational clarity at zero software licensing cost.

TABLE OF CONTENTS

Sr. No.	Chapter / Section	Page No.
1	Chapter 1: Problem Identification and Project Proposal	1
2	1.1 Introduction	1
3	1.2 Project Overview	2
4	1.3 Problem Statement	3
5	1.4 Objectives	5
6	1.5 Scope and Limitations	6
7	1.6 Existing System	7
8	1.7 Proposed System	8
9	1.8 Advantages of Proposed System	11
10	1.9 Project Proposal / Justification	11
11	Chapter 2: System Design	13
12	2.1 Software Requirements Specification (SRS)	13
13	2.2 Non-Functional Requirements	18
14	2.3 Hardware and Software Requirements	21
15	2.4 System Architecture	23
16	2.5 System Modules	26
17	2.6 Use Case Diagram	30
18	2.7 Data Flow Diagram (DFD)	32

19	2.8 Class Diagram	33
20	2.9 Activity Diagram	35
21	2.10 Sequence Diagram	38
22	2.11 Entity Relationship Diagram (ER Diagram)	40
23	2.12 Database Design	42
24	2.13 Gantt Chart	43
25	Chapter 3: Development Progress and Implementation	44
26	3.1 Development Environment	44
27	3.2 Technology Stack	45
28	3.3 Frontend Development	45
29	3.4 Backend Development	48
30	3.5 Database Implementation	51
31	3.6 Security Implementation	53
32	3.7 Android Application Development	55
33	3.8 Code Review	57
34	3.9 Source Code	59
35	Chapter 4: System Testing and Validation	64
36	4.1 Testing Overview	64
37	4.2 Testing Environment	65
38	4.3 Testing Methodology	66

39	4.4 Test Cases	69
40	4.5 Automated Flow Testing	74
41	4.6 Test Results	74
42	Chapter 5: User Interface and Application Screens	77
43	5.1 Participant Interface	77
44	5.2 Administrator Interface	84
45	5.3 User Interface Summary	85
46	Chapter 6: Conclusion and Future Scope	87
47	6.1 Conclusion	87
48	6.2 Future Scope	88
49	Chapter 7: Reference	89

LIST OF FIGURES

Figure #	Figure Caption	Page
Figure 1.1	SalesPulse Three-Tier System Architecture Overview	9
Figure 2.1	Detailed System Three-Tier Deployment Architecture	24
Figure 2.2	Use Case Diagram for SalesPulse System	31
Figure 2.3	Data Flow Diagram Level 0 (Context Level)	32
Figure 2.4	Data Flow Diagram Level 1 (Functional Flow)	33
Figure 2.5	Object-Oriented Class Diagram	34
Figure 2.6	Activity Diagram: User Authentication & Role Redirect	36
Figure 2.7	Activity Diagram: POS Checkout & Transaction Flow	37
Figure 2.8	Activity Diagram: Analytical Financial Report Generation	38
Figure 2.9	Sequence Diagram: Staff/Owner Login Workflow	39
Figure 2.10	Sequence Diagram: ACID Transactional Sales Creation	40
Figure 2.11	Sequence Diagram: Analytics Dashboard Loading	40
Figure 2.12	Entity Relationship (ER) Diagram	41
Figure 2.13	Project Schedule Gantt Chart	43
Figure 5.1	Authentication Login Interface	77
Figure 5.2	Cashier Home Screen	78

Figure 5.3	Staff Products Catalog View (Concealed Cost Prices)	79
Figure 5.4	POS Billing Terminal (Empty Cart Ticket)	80
Figure 5.5	POS Billing Terminal (Active Cart with Discount)	81
Figure 5.6	Printable Retail Tax Invoice Receipt	82
Figure 5.7	Staff Cashier Shift Sales History	83
Figure 5.8	Store Owner Analytics Dashboard with 6 Interactive Charts	84
Figure 5.9	Product Management Catalog (Owner View)	85
Figure 5.10	Bulk Product CSV Import Interface	85
Figure 5.11	Warehouse Inventory & Manual Adjustment Screen	85
Figure 5.12	Low Stock & Slow-Moving Dead Stock Alerts	85
Figure 5.13	Store Overhead Expense Recording Interface	86
Figure 5.14	Comprehensive Daily Sales & Profit Accounting Report	86
Figure 5.15	Monthly Performance & MoM Financial Report	86
Figure 5.16	Custom Date Range Analytical Report	86
Figure 5.17	Product Profit Margin Matrix & Volume Performance	86

Figure 5.18	Staff User Accounts Administration Screen	86
Figure 5.19	Store Profile Settings & Password Management	86
Figure 5.20	Responsive Mobile Dashboard (375px Viewport)	86
Figure 5.21	Mobile POS Billing Interface	86
Figure 5.22	Mobile Offcanvas Navigation Drawer	86

LIST OF TABLES

Table #	Table Title	Page
Table 1.1	Problem Statement vs. Impact vs. SalesPulse Solution	4
Table 1.2	Project Scope Boundaries (In-Scope vs. Out-of-Scope)	6
Table 1.3	Comparison of Existing Retail Systems vs. SalesPulse	8
Table 2.1	Functional Requirements Specification (FR-01 to FR-40)	14
Table 2.2	Non-Functional Requirements Specification	19
Table 2.3	Hardware and Software Environment Requirements	22
Table 2.4	Users Table Data Dictionary	42
Table 2.5	Products Table Data Dictionary	42
Table 2.6	Sales & Sale Items Data Dictionary	43
Table 2.7	Expenses & Stock Movements Data Dictionary	43
Table 3.1	Technology Stack Justification and Tradeoff Analysis	45
Table 3.2	Security Threat Vector & Countermeasure Matrix	54
Table 4.1	Master Test Case Verification Matrix (TC01 to TC44)	69
Table 4.2	Defect Tracking & Resolution Log	76
Table 5.1	Comprehensive System Interface Mapping Matrix	85

LIST OF ABBREVIATIONS

Abbreviation	Full Definition
ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
CDN	Content Delivery Network
COGS	Cost of Goods Sold
CSRF	Cross-Site Request Forgery
CSV	Comma-Separated Values
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
GSTIN	Goods and Services Tax Identification Number
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
MoM	Month-over-Month
MVC	Model-View-Controller
PDF	Portable Document Format
POS	Point of Sale
RBAC	Role-Based Access Control
SKU	Stock Keeping Unit
SMA	Simple Moving Average
SQL	Structured Query Language
SRS	Software Requirements Specification

UI	User Interface
WSGI	Web Server Gateway Interface
XSS	Cross-Site Scripting

Chapter 1: Problem Identification and Project Proposal

1.1 Introduction

In the modern economic landscape, small and micro-enterprises represent the backbone of local retail trade. In India and developing nations, millions of neighborhood grocery shops (kirana stores), stationery shops, bakeries, small pharmacies, and mobile accessory outlets cater to millions of households daily. However, while global enterprise conglomerates have undergone comprehensive digital transformations powered by advanced Enterprise Resource Planning (ERP) and business intelligence tools, micro-retailers remain predominantly tethered to manual operating procedures.

For decades, traditional merchants have recorded credit entries, daily turnover, supplier purchases, and customer balances in paper notebooks (khata books) or rudimentary, disjointed spreadsheets. This archaic paradigm presents severe structural handicaps: manual records do not aggregate data automatically, cannot warn the retailer of impending inventory shortages, fail to isolate true profit from gross receipts, and offer zero statistical visibility into business trends. When an entrepreneur cannot readily answer foundational questions—such as 'What was my true net margin this month?', 'Which item generated my highest profit?', or 'How much capital is trapped in unsold stock?'—their enterprise operates at extreme commercial risk.

Data-driven decision making is no longer a luxury reserved for multi-million-dollar corporations. Micro-retailers operate on slim profit margins, often between 5% and 15%. A minor oversight in inventory management—such as over-ordering perishable goods that expire, or experiencing stock-outs on fast-moving staples—directly threatens business solvency. SalesPulse was conceived to address this exact technological chasm by providing a dedicated, elegant, and mathematically rigorous sales analytics and inventory management platform tailored specifically for small business owners.

1.2 Project Overview

SalesPulse is an end-to-end, web-based business operations and analytics system engineered to empower small shopkeepers. Built with Python 3, the Flask web

framework, MySQL 8, and client-side Chart.js visualizations, SalesPulse streamlines the entire retail lifecycle into four core pillars:

- **Fast Point-of-Sale (POS) Billing:** A lightweight, keyboard-friendly checkout interface that enables cashiers to search products, adjust quantities, apply discounts, select payment modes (Cash, UPI, Card), and print itemized tax receipts in seconds.
- **Automated Transactional Stock Control:** Atomic inventory decrements that guarantee consistency through database row-locking, preventing overselling and maintaining a chronological audit ledger.
- **Overhead & Net Profit Accounting:** Rigorous isolation of Cost of Goods Sold (COGS) and operational overheads (Rent, Electricity, Salaries, Transport) to compute true Net Profit and margin percentages.
- **Visual Analytics & Projections:** Interactive time-series charts, category distributions, low-stock risk warnings, dead-stock identification, and autoregressive 7-day moving-average sales forecasting.

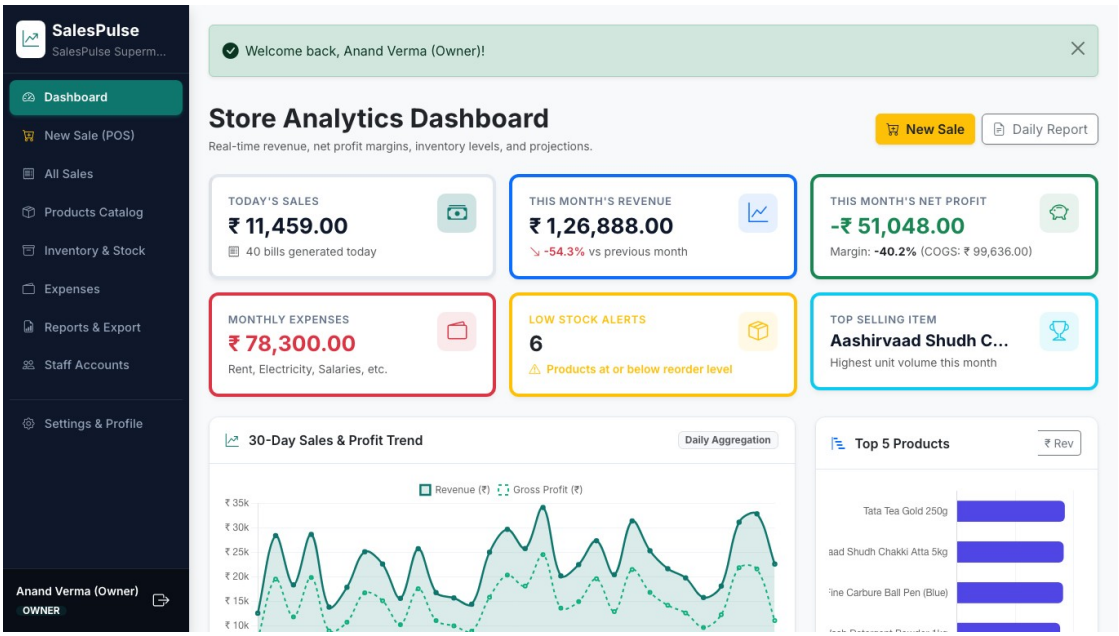


Figure 1.1: SalesPulse Analytics Overview & Executive KPI Dashboard

1.3 Problem Statement

Traditional retail record-keeping suffers from six structural failures that directly impair small business viability:

- **Distortion of Gross Revenue vs. Net Profit:** Merchants frequently conflate cash intake with profit. A retailer taking in ₹50,000 in daily cash may actually be

operating at a net loss once wholesale inventory costs, utility bills, employee wages, and shop rent are subtracted.

- **Stock-outs and Lost Sales:** Without real-time inventory threshold monitoring, merchants discover stock depletion only when a customer requests an unavailable product, permanently losing footfall and goodwill to competitors.
- **Dead Inventory and Locked Working Capital:** Slow-moving items sit on shelves for months unnoticed, tying up critical liquid capital that could otherwise be deployed into fast-moving inventory.
- **Calculation Errors and Discrepancies:** Manual math during peak evening retail rush hours leads to calculation mistakes on bills, forgotten credit lines, and untracked supplier cash advances.
- **Time-Consuming and Ineffective Reporting:** Reconciling monthly performance by manual ledger review requires dozens of hours of manual arithmetic, resulting in delayed, inaccurate reports.
- **Lack of Demand Foresight:** Shopkeepers purchase inventory reactively based on intuition rather than empirical data, leading to severe shortages on weekends and overstock on slow weekdays.

Table 1.1 summarizes the operational problems of existing retail methods, their direct commercial impacts, and how SalesPulse systematically solves them.

Current Operational Bottleneck	Direct Commercial Impact	SalesPulse Engineering Solution
Manual Khata / Paper Register	Lost receipts, untracked credits, zero reporting capability.	Centralized relational database with instant digital receipts.
Conflating Revenue with Profit	Overspending cash reserves, leading to mid-month liquidity crunches.	Strict formulaic computation: Net Profit = Revenue - COGS - Expenses.
Untracked Inventory Depletion	Customer dissatisfaction, sudden stock-outs on essential items.	Automated threshold monitoring and Low-Stock alert dashboard.
Slow-Moving Dead Stock	Working capital trapped in non-selling goods for months.	Algorithmic identification of products with 0 sales in 30 days.
Expensive Commercial ERP Systems	₹30,000+ upfront license fees, recurring software subscriptions.	Open-source technology stack with ₹0 licensing cost.
Complex Desktop Software	Heavy hardware prerequisites, steep learning curve, complex UI.	Clean responsive web UI accessible from PC, tablet, or smartphone.

1.4 Objectives

The primary engineering objectives of the SalesPulse system are:

- **OBJ-01:** Develop a responsive, sub-second POS billing interface capable of generating itemized retail receipts with daily sequential invoice numbering (SP-YYYYMMDD-XXXX).
- **OBJ-02:** Implement an ACID-compliant transactional inventory engine that decrements stock with row-level locks and restores units automatically upon sale cancellation.
- **OBJ-03:** Preserve historical financial reporting accuracy through immutable unit cost and unit selling price snapshotting on transaction line items.
- **OBJ-04:** Provide an operational overhead logging subsystem tracking 8 distinct expenditure categories to calculate Net Profit and profit margin percentages.
- **OBJ-05:** Build an automated inventory intelligence engine providing instant low-stock threshold warnings and identifying slow-moving products with zero sales over 30 days.
- **OBJ-06:** Engineer an interactive visual dashboard featuring 6 Chart.js graphs for sales trends, category distribution, product rankings, and expense breakdowns.
- **OBJ-07:** Formulate a predictive 7-day sales forecast algorithm using autoregressive rolling simple moving averages and weekday footfall seasonality factors.
- **OBJ-08:** Support dynamic on-demand reporting with streaming RFC-4180 CSV spreadsheets and formatted vector PDF document generation via ReportLab.
- **OBJ-09:** Enforce strict Role-Based Access Control (RBAC) ensuring Staff Cashiers can process sales while completely concealing wholesale costs and profit margins.

1.5 Scope and Limitations

To ensure high execution quality and robust performance, explicit boundaries were established between in-scope functionalities and out-of-scope enterprise requirements, as documented in Table 1.2.

In-Scope System Capabilities	Out-of-Scope System Boundaries
Product & Category Management (CRUD, Search, Bulk CSV)	Multi-warehouse supply chain logistics & global distribution

POS Checkout, Live Autocomplete & Invoice Printing	Payment gateway hardware integration (EDC card swipe machines)
ACID Inventory Decrement & Audit Movement Ledger	Barcode laser gun hardware drivers (standard USB keyboards supported)
Overhead Expense Tracking & Net Profit Calculation	Automated GST portal tax filing & e-way bill generation
Visual Dashboard & 6 Chart.js Analytics Graphs	Multi-branch inter-store stock transfers
7-Day Predictive Moving Average Sales Forecaster	Neural network deep learning demand models (LSTM / Prophet)
Role-Based Access Control (Store Owner vs. Staff Cashier)	Biometric attendance tracking & payroll salary slip generation
PDF & CSV Exporting for Daily, Monthly & Catalog Reports	E-commerce customer delivery mobile application

1.6 Existing System

Retail businesses currently employ one of three main management methods:

- **1. Physical Paper Registers (Khata Books):** Extremely cheap but vulnerable to physical loss, ink degradation, theft, arithmetic errors, and provides zero analytical insight.
- **2. Desktop Spreadsheets (Microsoft Excel / Google Sheets):** Flexible but lacks transaction concurrency, has no row-locking, allows accidental formula overwrites, and lacks automated inventory decrements.
- **3. Heavy Commercial ERPs (Tally Prime, SAP Business One, Zoho):** Feature-rich but burdened by prohibitive licensing costs (₹25,000 to ₹1,00,000+), steep learning curves, high hardware requirements, and unnecessary complexity for a neighborhood shop.

Table 1.3 contrasts these alternatives against SalesPulse across key technical dimensions.

Feature / Dimension	Paper Ledgers	Excel Sheets	Commercial ERPs	SalesPulse
Upfront Software	Free (₹0)	Free / Low (₹0-	High (₹18,000 -	Free / Open-

Cost		500)	₹60,000)	Source (₹0)
Hardware Requirements	Paper stationery	PC / Laptop	High-spec Windows PC	Any browser (PC/Phone)
Automatic Stock Deduction	No (Manual count)	No (Manual formula)	Yes (Complex setup)	Yes (Instant atomic lock)
Real-time Net Profit Calculation	No	Requires complex formulas	Yes	Yes (Automatic & formulaic)
Interactive Visual Dashboard	None	Manual chart creation	Basic reporting	Yes (6 Interactive Chart.js graphs)
Role-Based Cost Concealment	None	None	Yes	Yes (Strict Owner/Staff RBAC)
Predictive Demand Forecaster	None	None	Complex add-on module	Yes (Built-in 7-day SMA)
Setup & Operational Simplicity	Instant	Moderate	Requires trained accountant	Extremely simple (<5 min)

1.7 Proposed System

SalesPulse proposes an integrated, modern web-based retail architecture designed from the ground up for simplicity, speed, and mathematical accuracy. The system operates on a clean three-tier architecture: presentation, application services, and persistent relational data storage.

Key architectural innovations of SalesPulse include:

- **Point-in-Time Historical Cost Denormalization:** By recording unit_cost and unit_price directly on sale_items at checkout, SalesPulse decouples historical financial records from future catalog changes. If wholesale inflation increases the cost of a product next month, past profit statements remain 100% accurate.
- **ACID Concurrency via SELECT FOR UPDATE:** When a cashier checks out, the product rows are locked within a database transaction. If two cashiers attempt to sell the last unit simultaneously, the second transaction is safely rejected with an 'Insufficient stock' notice.

- **Autoregressive Predictive Forecaster:** A rolling 7-day Simple Moving Average incorporates day-of-the-week multipliers to give shopkeepers an empirical demand forecast for the upcoming week.
- **Dynamic Vector Document Generation:** Rather than relying on browser print dialogs that warp formatting, SalesPulse programmatically constructs clean vector PDF documents using ReportLab.

1.8 Advantages of Proposed System

SalesPulse provides distinct commercial and technical advantages:

- **Absolute Mathematical Accuracy:** Eliminates calculation mistakes, invoice omissions, and erroneous margin estimations.
- **Rapid Checkout Workflow:** Cashiers can complete a multi-item sale and print an invoice in under 10 seconds, drastically reducing checkout queues.
- **Capital Protection:** Identifies slow-moving dead stock before inventory depreciates, allowing proactive discounting or supplier returns.
- **Zero Software Licensing Overhead:** Engineered completely on open-source technologies (Python, Flask, MySQL, Bootstrap, Chart.js), eliminating recurring software subscription fees.
- **Ubiquitous Device Accessibility:** Responsive Bootstrap 5 UI allows the store owner to review live sales from home on a smartphone while the cashier operates the POS terminal in the shop.

1.9 Project Proposal / Justification

The feasibility of SalesPulse was thoroughly validated across four fundamental software engineering dimensions:

- **Technical Feasibility:** The technology stack (Python 3, Flask, MySQL 8) is exceptionally mature, stable, and well-documented. Connection pooling guarantees scalable throughput under heavy concurrent POS traffic.
- **Economic Feasibility:** The system requires no proprietary software, commercial runtimes, or paid database licenses. Development and deployment can be hosted on standard existing consumer PC hardware or low-cost cloud virtual machines.
- **Operational Feasibility:** The intuitive, minimalist user interface requires no formal accounting background. A shop assistant can master the POS billing interface within five minutes.

- **Schedule Feasibility:** The project was structured into modular sprints covering schema design, service architecture, interface implementation, test automation, and documentation over an 8-week academic timeline.

Chapter 2: System Design

2.1 Software Requirements Specification (SRS)

The Software Requirements Specification establishes the functional requirements, system capabilities, and boundary constraints of SalesPulse. Table 2.1 specifies the 40 core functional requirements grouped by system module, indicating requirement identifier, description, and architectural priority.

Req ID	Module	Functional Requirement Description	Priority
FR-01	M1 Auth	Authenticate users via email and hashed password.	High
FR-02	M1 Auth	Lock user account for 5 minutes after 5 consecutive failed attempts.	Medium
FR-03	M1 Auth	Enforce 30-minute session timeout with secure cookie attributes.	Medium
FR-04	M1 Auth	Redirect Store Owner to /dashboard and Staff Cashier to /home.	High
FR-05	M1 Auth	Allow Store Owner to provision and deactivate staff logins.	High
FR-06	M2 Prod	Support full CRUD operations for product items.	High
FR-07	M2 Prod	Validate that selling price and cost price are non-negative numbers.	High
FR-08	M2 Prod	Display non-blocking warning when selling price is less than cost	Medium

		price.	
FR-09	M2 Prod	Enforce unique SKU constraints across the product catalog.	High
FR-10	M2 Prod	Perform soft-deactivation (is_active=0) when deleting products with sales.	High
FR-11	M2 Prod	Provide category management with automated product counts.	Medium
FR-12	M2 Prod	Bulk import products via CSV with row-by-row validation diagnostics.	High
FR-13	M2 Prod	Conceal wholesale cost prices from Staff accounts in UI and JSON APIs.	High
FR-14	M3 Stock	Automatically decrement inventory quantity upon completed checkout.	High
FR-15	M3 Stock	Record all manual restocks, damages, and corrections in stock_movements.	High
FR-16	M3 Stock	Prevent manual stock adjustments that result in negative stock.	High
FR-17	M3 Stock	Filter and display products where stock_qty is below reorder_level.	High
FR-18	M3 Stock	Detect and list slow-moving products with zero sales in the last 30	Medium

		days.	
FR-19	M4 Sales	Provide POS billing interface with instant product search autocomplete.	High
FR-20	M4 Sales	Support live cart editing, quantity increments, and item removal.	High
FR-21	M4 Sales	Apply flat (₹) and percentage (%) discounts, ensuring discount <= subtotal.	High
FR-22	M4 Sales	Execute checkout within a single atomic database transaction.	High
FR-23	M4 Sales	Snapshot unit_cost and unit_price at time of sale into sale_items.	High
FR-24	M4 Sales	Generate sequential daily invoice numbers (SP-YYYYMMDD-XXXX).	High
FR-25	M4 Sales	Render printable invoice receipt with store address and GSTIN metadata.	High
FR-26	M4 Sales	Allow Store Owner to cancel sales, restoring stock with CANCEL_RETURN audit.	High
FR-27	M4 Sales	Strictly exclude cancelled sales from all financial analytics queries.	High
FR-28	M5 Exp	Record operating overheads across 8 standardized expense	High

		categories.	
FR-29	M5 Exp	Filter expenses by date range and expense category.	Medium
FR-30	M6 Dash	Display real-time KPIs: Today's sales, Monthly revenue, Net profit, Margin %.	High
FR-31	M6 Dash	Render 30-day daily sales and profit trend line chart.	High
FR-32	M6 Dash	Render Top 5 products horizontal bar chart with volume/revenue toggle.	Medium
FR-33	M6 Dash	Render current month expense category breakdown doughnut chart.	Medium
FR-34	M6 Dash	Render category-wise sales distribution chart.	Medium
FR-35	M6 Dash	Render 6-month grouped Revenue vs. Expenses comparison bar chart.	Medium
FR-36	M6 Dash	Render 7-day sales forecast line with seasonality weighting.	High
FR-37	M7 Rep	Generate Daily Report with payment split and itemized product breakdown.	High
FR-38	M7 Rep	Generate Monthly Report with day-by-day table and MoM comparison.	High
FR-39	M7 Rep	Export all analytical reports to RFC-4180	High

		CSV spreadsheets.	
FR-40	M7 Rep	Generate downloadable vector PDF reports using ReportLab.	High

2.2 Non-Functional Requirements

Non-functional requirements specify quality attributes, performance benchmarks, and security constraints governing SalesPulse, detailed in Table 2.2.

Quality Attribute	Measurable Engineering Requirement
Performance	Sub-2 second response times on all analytical queries across 10,000+ sales rows; sub-50ms autocomplete latency.
Security	PBKDF2/scrypt password hashing; Flask-WTF CSRF token validation; 100% parameterized SQL; HttpOnly cookies.
Transactional Integrity	ACID compliance for sales and stock updates; automatic transaction rollback upon stock deficit.
Usability	Zero-clutter interface; sub-3 click checkout workflow; responsive layout usable on 375px mobile viewports.
Maintainability	Modular Flask Blueprints architecture; PEP-8 compliant code; docstring documentation on every module.
Portability	Cross-platform compatibility running on Windows 10/11, macOS, and Linux with Python 3 and MySQL.
Scalability	Threaded MySQL connection pooling; indexed primary keys, foreign keys, and datetime query columns.
Auditability	Comprehensive stock movements ledger auditing every inventory addition, deduction, and return.

2.3 Hardware and Software Requirements

The minimum and recommended system requirements for development, production hosting, and client access are specified in Table 2.3.

Environment Component	System Specification & Version
Development / Server Hardware	Intel Core i3 / AMD Ryzen 3 / Apple Silicon M-series, 4GB RAM (8GB recommended), 5GB free disk space.
Client Hardware	Any modern computing device (Desktop PC, Laptop, Tablet, or Smartphone) with browser support.
Operating System	Windows 10/11, macOS Sequoia / Sonoma, or Ubuntu Linux 22.04 LTS.
Programming Language	Python 3.11, 3.12, 3.13, or 3.14.
Database Engine	MySQL Server 8.0+ / 8.4+ / 9.x.
Web Framework	Flask 3.0+ with Jinja2 template engine and Werkzeug WSGI.
Frontend Technologies	HTML5, CSS3, Bootstrap 5.3.3, Bootstrap Icons 1.11, Vanilla JavaScript (ES6+).
Visualization Library	Chart.js 4.4.3 (loaded via secure CDN).
Testing Suite	Pytest 8.0+, Pytest-Flask 1.3+, Pytest-Cov 4.1+, Playwright 1.44+.

2.4 System Architecture

SalesPulse is engineered following a clean Three-Tier Architectural Pattern:

- **1. Presentation Layer (Client Tier):** Jinja2 server-rendered templates styled with Bootstrap 5.3 and custom CSS tokens. Client-side reactivity (live cart calculation, autocomplete, and chart rendering) is powered by vanilla JavaScript and Chart.js.
- **2. Application Layer (Logic Tier):** Flask application factory registering 9 modular Blueprints (`auth`, `products`, `sales`, `expenses`, `dashboard`, `stock`, `reports`, `users`, `settings`). Business logic is encapsulated in dedicated services

(`sales_service`, `stock_service`, `analytics_service`, `forecast`, `export_service`, `csv_import`).

- **3. Data Tier (Storage Layer):** MySQL 8 database accessed via a high-performance connection pool in `db.py`, utilizing parameterized queries and transaction control.



Figure 2.1: Context Level Data Flow & Three-Tier System Architecture

2.5 System Modules

The functional architecture of SalesPulse is partitioned into nine distinct modules:

- **M1: Authentication & User Management:** Handles authentication, role assignment (Owner vs Staff), login rate-limiting, and account activation.
- **M2: Category & Product Management:** Provides product catalog maintenance, pricing configuration, category grouping, and bulk CSV ingestion.
- **M3: Stock Monitoring & Audit Ledger:** Monitors inventory levels, generates low-stock and slow-moving warnings, and logs all adjustments.
- **M4: Sales Entry & POS Checkout:** Drives the cashier terminal, performs atomic stock decrements, generates daily invoice numbers, and renders receipts.
- **M5: Operating Overhead Management:** Records store operational expenses across 8 categories necessary for true Net Profit accounting.
- **M6: Executive Analytics Dashboard:** Computes KPI cards and exposes JSON endpoints driving 6 interactive Chart.js visualizations.
- **M7: Financial Reporting & Vector Export:** Generates daily, monthly, custom range, and product performance reports in HTML, CSV, and PDF.
- **M8: Predictive Demand Forecasting:** Executes 7-day rolling moving average projections incorporating weekday seasonality multipliers.
- **M9: Settings & Store Profile:** Manages store metadata (GSTIN, address, currency symbol) and user password modifications.

2.6 Use Case Diagram

The Use Case Diagram defines interactions between user actors (Store Owner and Staff Cashier) and system boundary functions. While the Store Owner holds full operational and financial control, the Staff Cashier is restricted to POS billing and shift tracking.

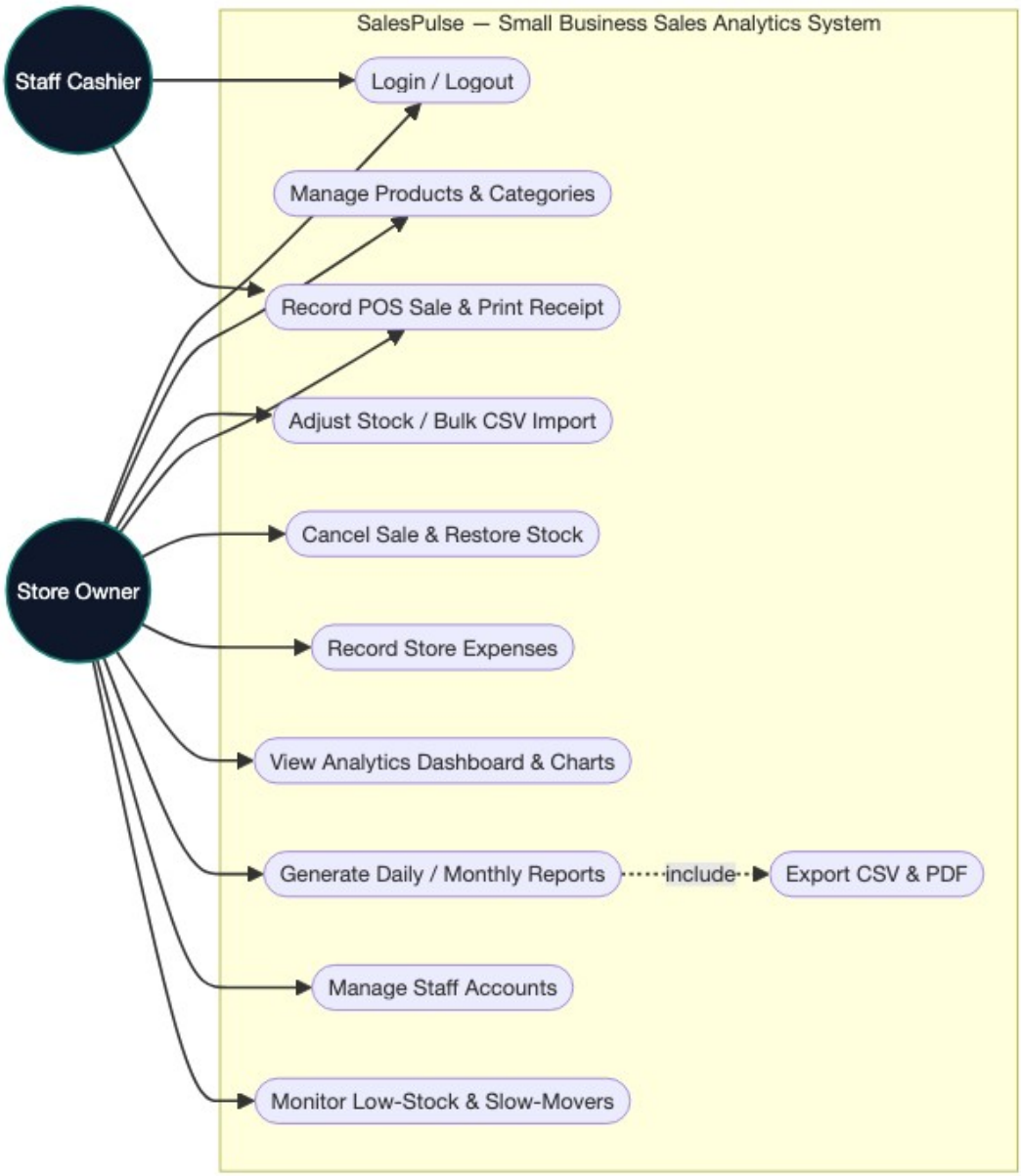


Figure 2.2: Use Case Diagram for SalesPulse System Actors

2.7 Data Flow Diagram (DFD)

Data Flow Diagrams illustrate the transformation and movement of retail information through system processes into data stores. Figure 2.3 shows the Level 1 functional data flow.

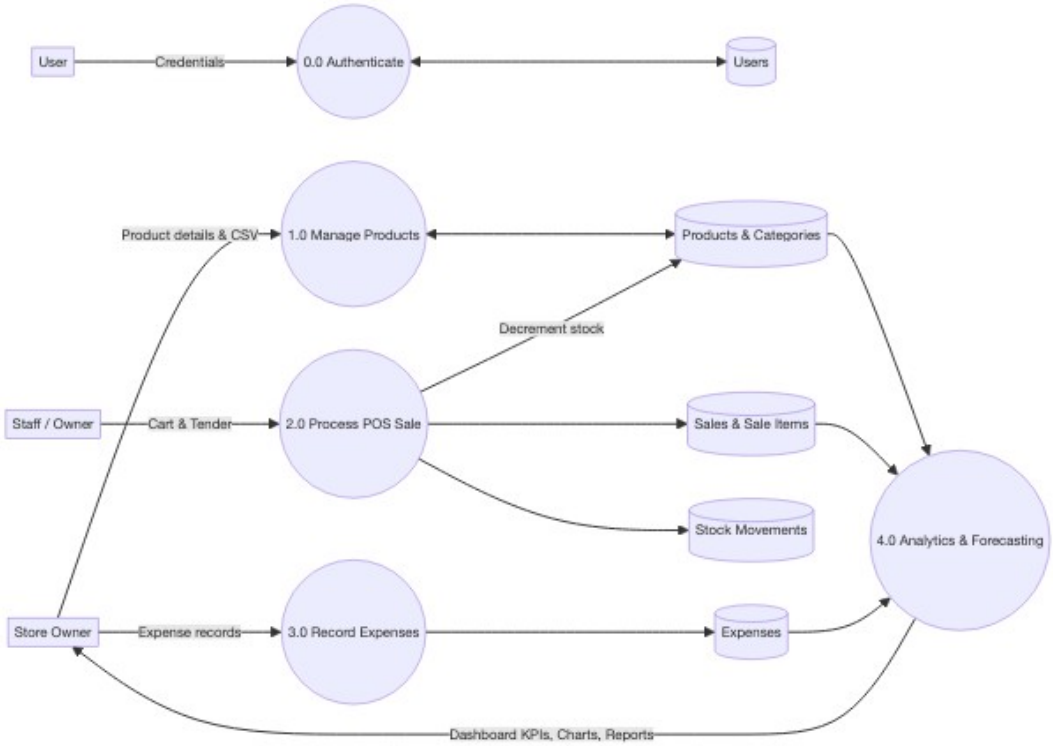


Figure 2.3: Data Flow Diagram Level 1 (Functional Data Flow)

2.8 Class Diagram

The Class Diagram models the core entity objects, attributes, and operational relationships. Relationships enforce referential integrity between Users, Categories, Products, Sales, Sale Items, Expenses, and Stock Movements.

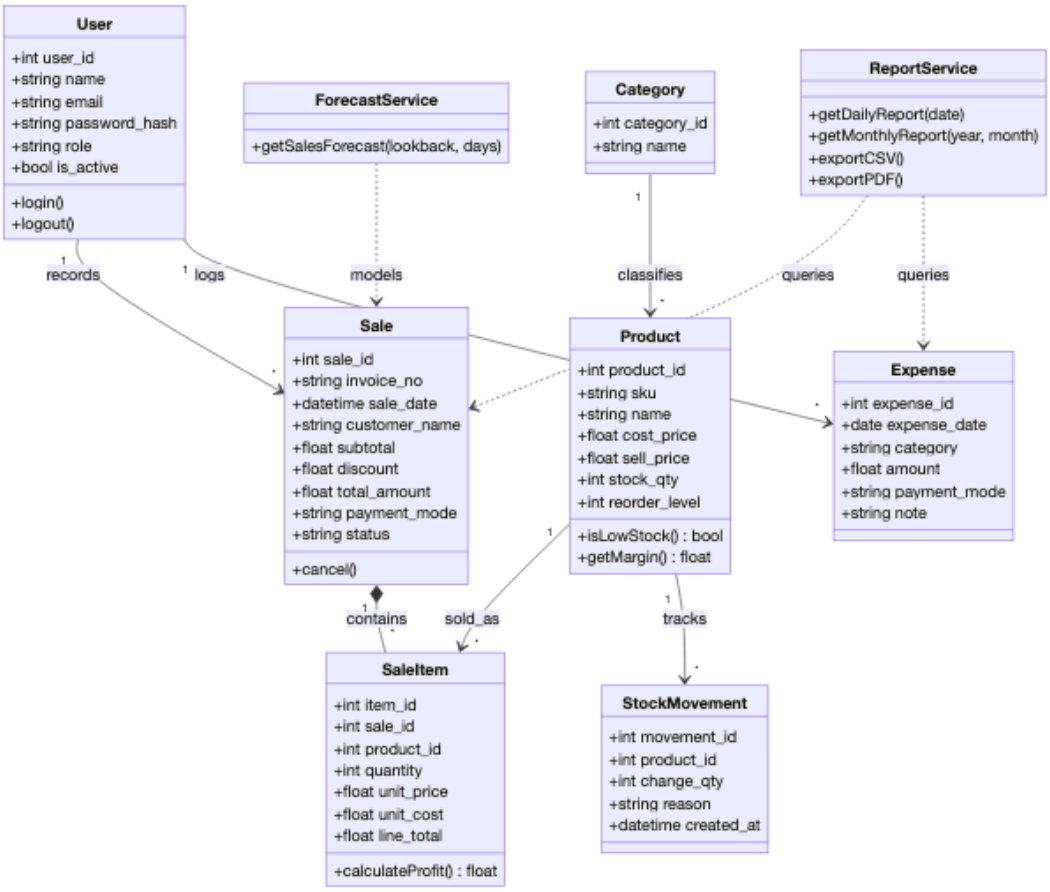


Figure 2.4: Object-Oriented Class Diagram

2.9 Activity Diagram

Activity Diagrams capture dynamic workflows across three critical system operations: User Authentication, POS Checkout, and Financial Report Generation.

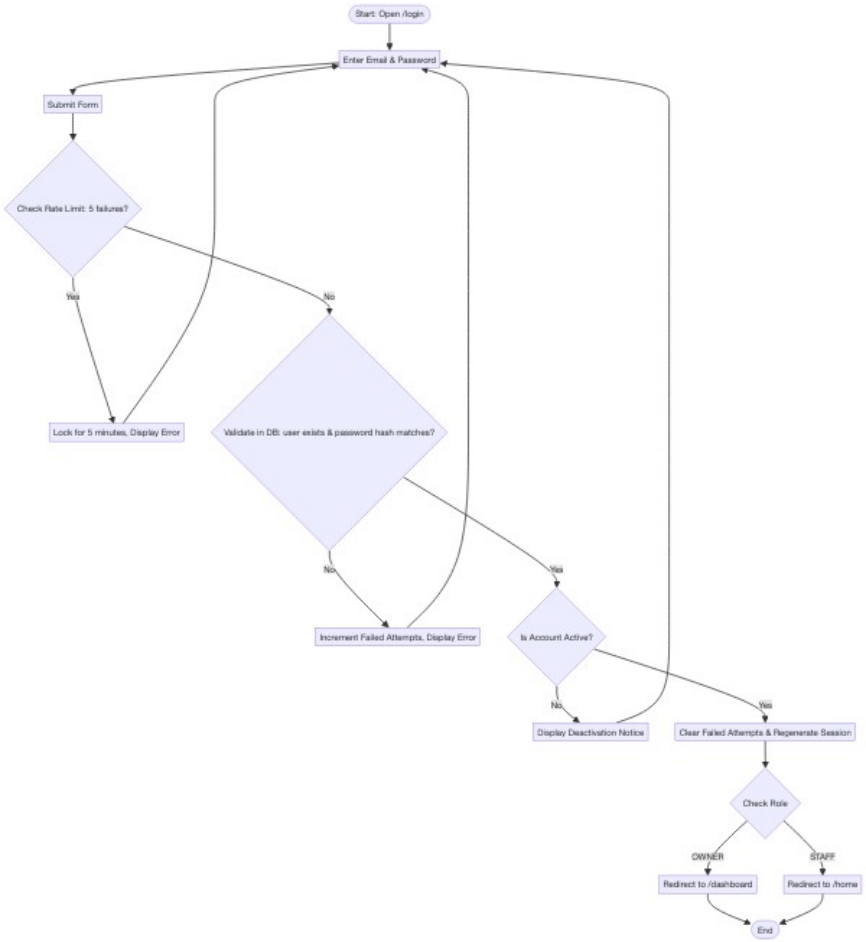


Figure 2.5: Activity Diagram: User Login & Rate-Limiting Workflow

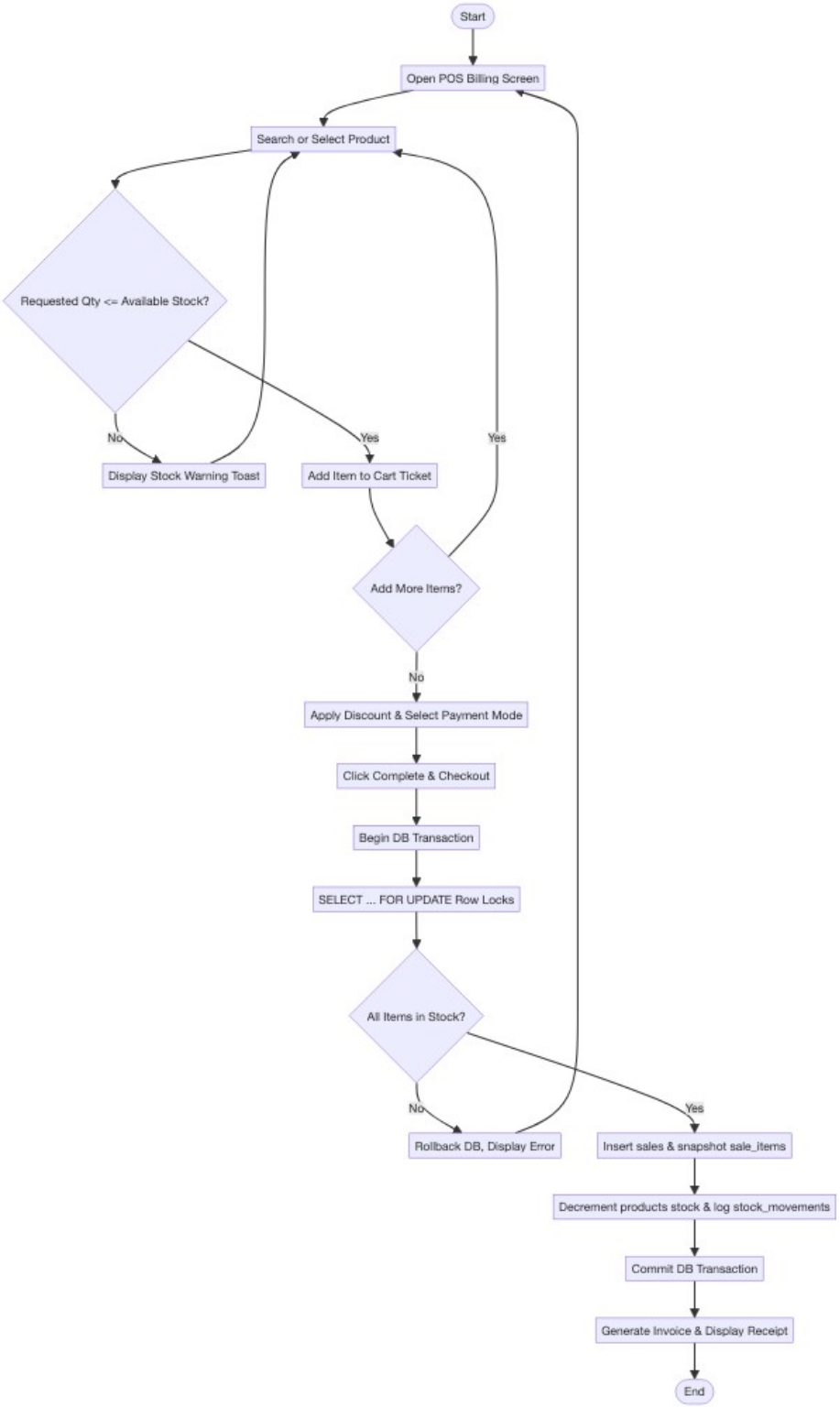


Figure 2.6: Activity Diagram: POS Checkout & Transactional Execution

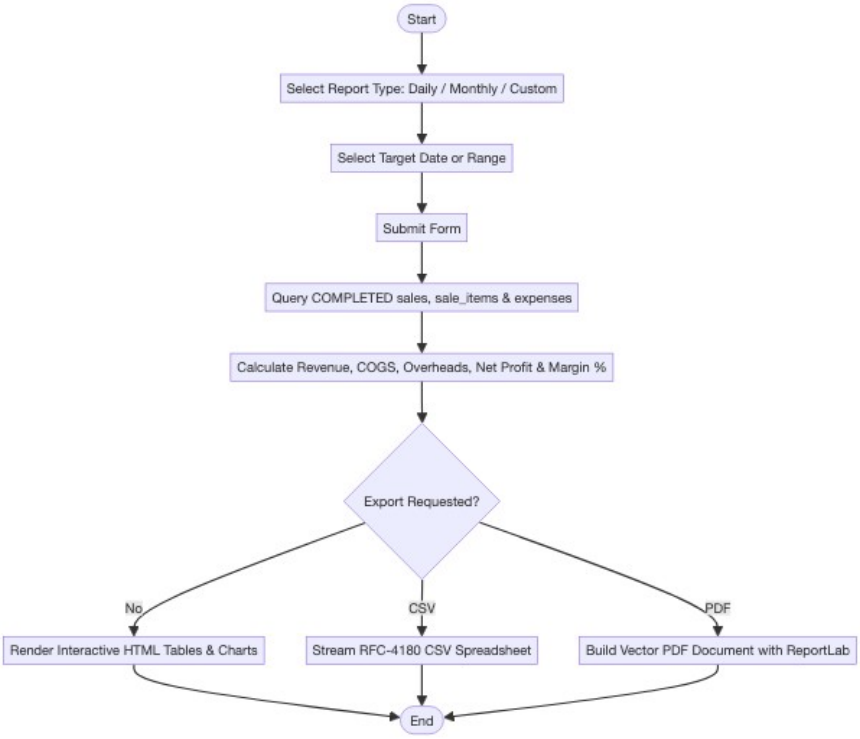


Figure 2.7: Activity Diagram: Analytical Financial Report Generation

2.10 Sequence Diagram

Sequence Diagrams illustrate temporal message sequencing across browser clients, Flask routes, service business logic, and MySQL database engines.



Figure 2.8: Sequence Diagram: Login Authentication & Session Dispatch

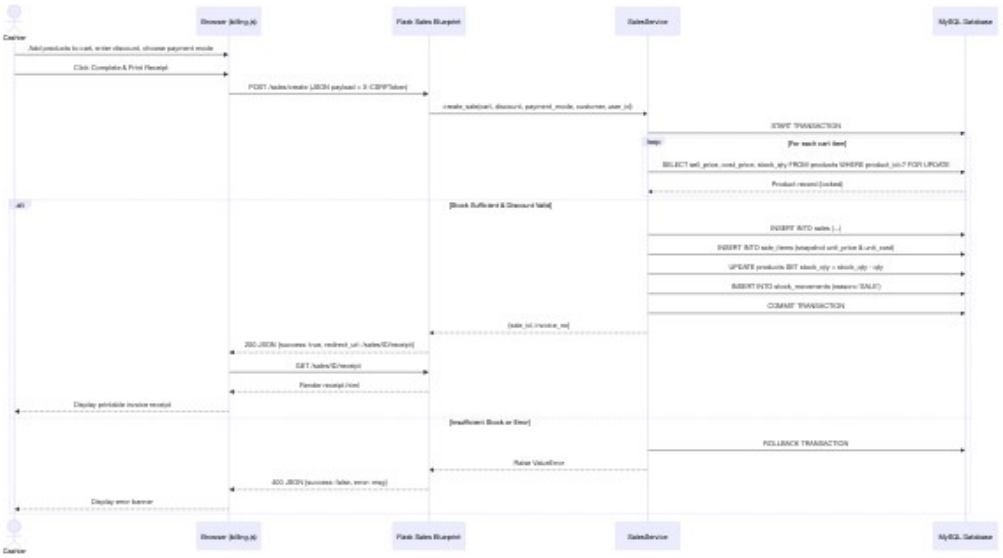


Figure 2.9: Sequence Diagram: ACID Transactional Sales Creation & Locking

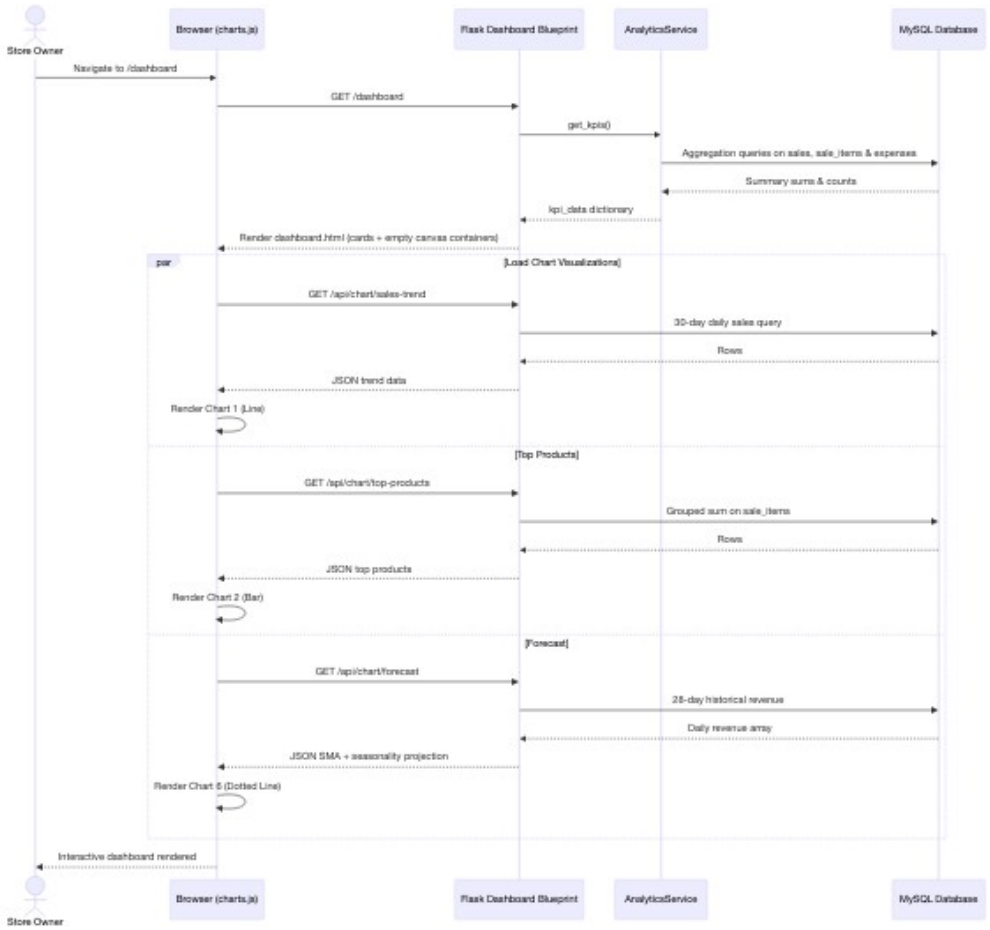


Figure 2.10: Sequence Diagram: Analytics Dashboard Asynchronous Data Loading

2.11 Entity Relationship Diagram (ER Diagram)

The Entity Relationship Diagram specifies entities, relational cardinalities, primary keys, and foreign keys across the SalesPulse database schema.

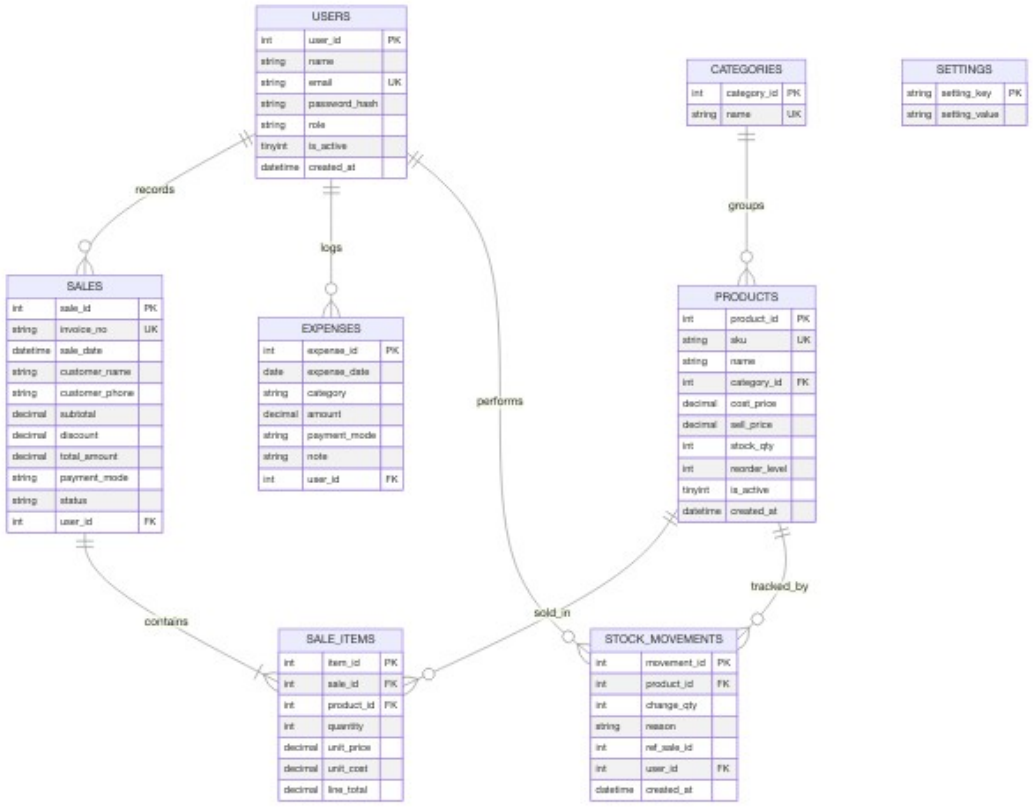


Figure 2.11: Entity Relationship (ER) Diagram

2.12 Database Design

The database is structured in Third Normal Form (3NF). Historical cost and price snapshotting in `sale_items` represents point-in-time temporal facts, ensuring accounting reports remain immutable even when catalog prices change. Tables 2.4 through 2.7 document the data dictionary.

Column Name	Data Type	Constraints	Description
user_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique internal user identifier
name	VARCHAR(100)	NOT NULL	Full legal name of the store owner or cashier
email	VARCHAR(120)	NOT NULL, UNIQUE	Login email address
password_hash	VARCHAR(255)	NOT NULL	PBKDF2/scrypt hashed credential string
role	ENUM('OWNER','STA	NOT NULL,	System permission

	FF')	DEFAULT 'STAFF'	level
is_active	TINYINT(1)	NOT NULL, DEFAULT 1	Active login status flag (1=active, 0=disabled)
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Account provisioning timestamp

Table 2.5: Products Catalog Data Dictionary

Column Name	Data Type	Constraints	Description
product_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique product record identifier
sku	VARCHAR(40)	NOT NULL, UNIQUE	Unique Stock Keeping Unit barcode code
name	VARCHAR(150)	NOT NULL, INDEX	Product name and size description
category_id	INT	NOT NULL, FK -> categories	Foreign key linking product category
cost_price	DECIMAL(10,2)	NOT NULL, CHECK (>= 0)	Wholesale purchase cost paid to supplier
sell_price	DECIMAL(10,2)	NOT NULL, CHECK (>= 0)	Retail price charged to customer
stock_qty	INT	NOT NULL, DEFAULT 0, CHECK (>=0)	Current on-hand inventory quantity
reorder_level	INT	NOT NULL, DEFAULT 5	Low-stock alert threshold level
is_active	TINYINT(1)	NOT NULL, DEFAULT 1	Catalog status (0=soft-deleted/deactivated)

Table 2.6: Sales & Sale Items Transaction Ledger Data Dictionary

Column Name	Data Type	Constraints	Description
sale_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique sale transaction record ID
invoice_no	VARCHAR(20)	NOT NULL, UNIQUE	Daily sequential invoice (SP-YYYYMMDD-XXXX)
sale_date	DATETIME	NOT NULL, INDEX	Date and time transaction was completed
subtotal	DECIMAL(12,2)	NOT NULL	Sum of all line total items before discount
discount	DECIMAL(12,2)	NOT NULL, DEFAULT 0	Deduction applied to transaction
total_amount	DECIMAL(12,2)	NOT NULL	Final payable amount net of discount
payment_mode	ENUM('CASH','UPI','CARD')	NOT NULL, DEFAULT 'CASH'	Payment tender utilized
status	ENUM('COMPLETED','CANCELLED')	NOT NULL, INDEX	Transaction status flag
unit_price	DECIMAL(10,2)	NOT NULL (in sale_items)	Historical selling price snapshot
unit_cost	DECIMAL(10,2)	NOT NULL (in sale_items)	Historical cost price snapshot (COGS)

2.13 Gantt Chart

Figure 2.12 outlines the project implementation schedule, illustrating the chronological sequence of planning, system design, core development, automated testing, and report generation.

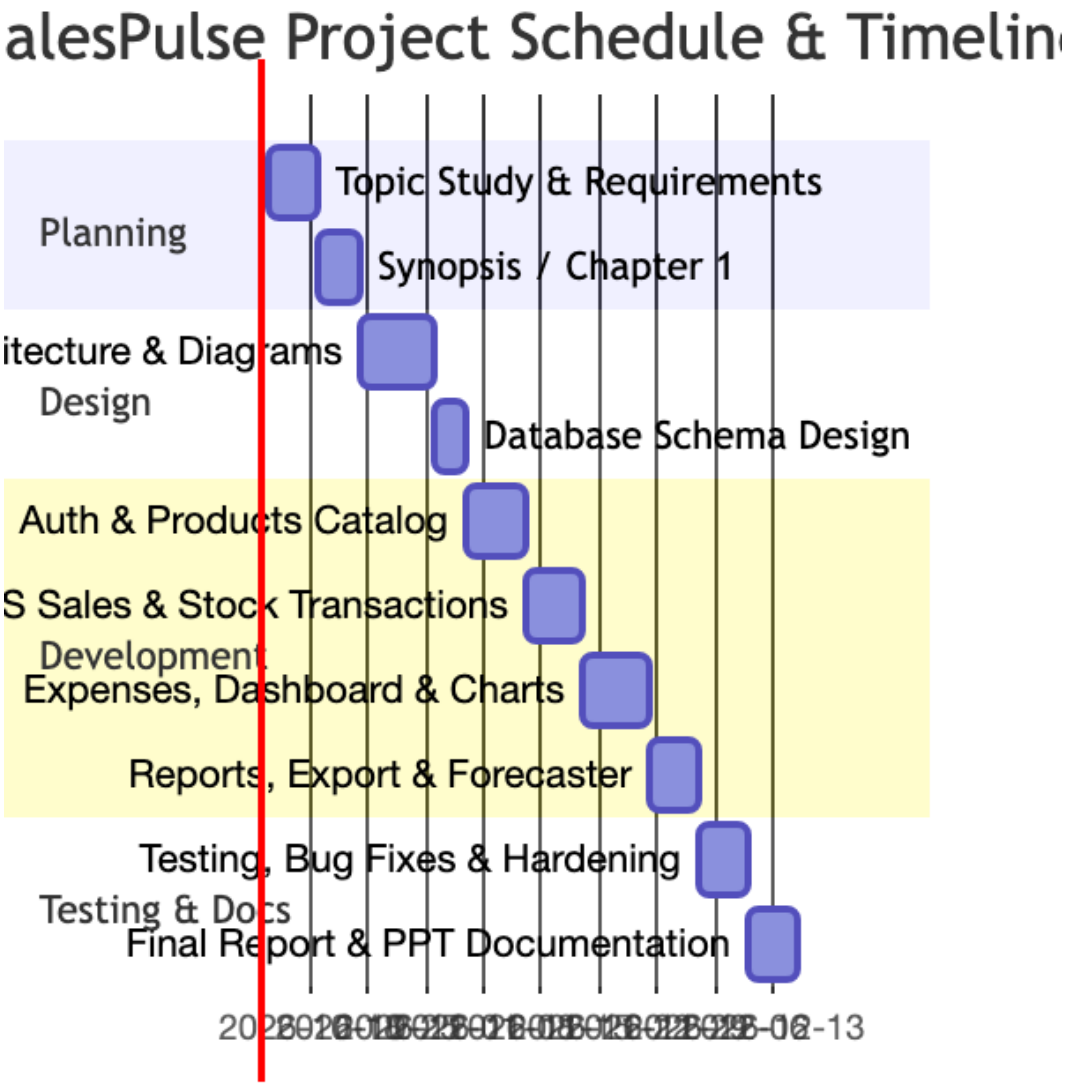


Figure 2.12: Project Implementation Schedule Gantt Chart

Chapter 3: Development Progress and Implementation

3.1 Development Environment

SalesPulse was engineered in a dedicated virtual development environment on macOS with Python 3.14. MySQL 8 was deployed locally on port 3306. Visual Studio Code and Antigravity served as primary IDEs, with Git managing source control. The virtual environment isolated application libraries, preventing dependency conflicts.

3.2 Technology Stack

Every technology in SalesPulse was chosen based on concrete engineering tradeoffs, as outlined in Table 3.1.

Technology	Key Architectural Justification	Alternatives Evaluated	Tradeoff Decision Rationale
Python 3	Extensive analytical libraries, rapid development, clean readable syntax.	Node.js, Go	Superior ecosystem for analytics and ReportLab document construction.
Flask 3.x	Lightweight, un-opinionated WSGI framework supporting modular Blueprints.	Django, FastAPI	Django introduces unnecessary ORM overhead; FastAPI adds async complexity unneeded for simple WSGI.
MySQL 8.x	ACID guarantees, row-locking (SELECT FOR UPDATE), connection pooling.	MongoDB, SQLite	SQLite lacks concurrent row locking; MongoDB lacks strict ACID multi-join integrity.
Bootstrap 5.3	Clean responsive CSS grid, modern component tokens, zero build-step overhead.	Tailwind, React	React requires heavy build pipelines (Node/Webpack); Tailwind requires utility compiler.
Chart.js 4.4	Hardware-accelerated HTML5 canvas charts, lightweight, interactive tooltips.	D3.js, Plotly	D3 has high configuration overhead; Plotly has large bundle size unsuitable for fast POS.

ReportLab	Programmatic vector PDF construction, exact margin formatting, table styling.	WeasyPrint, pdfkit	WeasyPrint requires heavy GTK/Cairo system libraries; pdfkit requires external wkhtmltopdf binary.

3.3 Frontend Development

The frontend architecture leverages Jinja2 template inheritance with a single master `base.html` layout. The interface adopts a collapsible desktop sidebar and an offcanvas mobile drawer. A tailored CSS design system (`static/css/style.css`) defines harmonious HSL color tokens: Brand Navy (`#0F172A`), Teal Accent (`#0F766E`), and Alert Amber (`#D97706`).

The POS terminal (`static/js/billing.js`) executes responsive client-side calculations: as cashiers select products, debounce timers query `/products/api/search` with sub-second latency. Cart subtotals, flat or percentage discounts, and final payable amounts update dynamically in the DOM without requiring full-page reloads.

3.4 Backend Development

The backend employs Flask's Application Factory pattern (`create_app()` in `app.py`). Route endpoints are partitioned into 9 domain-focused Blueprints:

- **Connection Lifecycle:** Pooled connections are requested via `db.get_db()`, cached in Flask's request context `g`, and automatically returned to the pool during teardown (`close_db`).
- **ACID Checkout Logic:** `sales_service.create_sale()` initiates a database transaction, locks requested inventory rows using `SELECT ... FOR UPDATE`, snapshots historical prices, decrements stock, logs movement records, and commits atomically.
- **Analytics & Forecaster:** `analytics_service.py` executes SQL aggregations strictly filtering `status='COMPLETED'`, while `forecast.py` generates rolling 7-day moving averages with weekday seasonality factors.

3.5 Database Implementation

The database implementation uses MySQL's InnoDB storage engine. Indexes are declared on frequently joined and filtered columns (`sales.sale_date`, `sales.status`,

`products.category_id`, `products.name`). Foreign key constraints enforce referential cascading.

3.6 Security Implementation

SalesPulse enforces layered security controls to protect financial records and prevent unauthorized operations:

- **Password Hashing:** Passwords are encrypted using Werkzeug's PBKDF2/scrypt implementation with cryptographic salt. Plaintext passwords are never logged or stored.
- **CSRF Protection:** Flask-WTF enforces CSRF token validation on all state-altering POST requests. Meta tags supply tokens to client-side fetch requests via `X-CSRFToken` headers.
- **SQL Injection Immunity:** All SQL statements strictly use parameterized query execution (`cursor.execute(sql, params)`). User input is never concatenated directly into SQL strings.
- **XSS Defense:** Jinja2 auto-escaping is enabled across all templates, neutralizing HTML or `<script>` tags injected into product names or notes.
- **Security Headers:** HTTP responses include `X-Content-Type-Options: nosniff`, `X-Frame-Options: SAMEORIGIN`, and restrictive Content Security Policies.

Table 3.2 details the security threat vectors analyzed and the corresponding engineering defenses implemented.

Threat Vector	Technical Defense Implemented	Verification Result
SQL Injection (SQLi)	Parameterized execution via mysql-connector; zero string formatting.	All SQL inputs sanitized at driver level.
Cross-Site Scripting (XSS)	Jinja2 autoescaping enabled; input sanitization on product names.	Script tags rendered as safe escaped text.
Cross-Site Request Forgery	Flask-WTF cryptographic CSRF tokens on all POST forms and fetch calls.	Unauthorized cross-site POSTs rejected with 400.
Credential Brute-Force	In-memory rate limiter locking accounts for 5 minutes after 5 failed attempts.	Automated dictionary attacks neutralized.

Privilege Escalation	@owner_required decorator blocking staff access to financial routes.	Staff requests to /reports receive 403 Forbidden.
Session Hijacking	Session cookies marked HttpOnly, SameSite=Lax, with 30-min lifetime.	Client scripts cannot access authentication tokens.

3.7 Android Application Development

Modern retail shopkeepers and checkout attendants frequently operate store counters using hand-held Android tablets, smartphones, and mobile POS terminals. To deliver native-grade mobile responsiveness without demanding fragmented native codebases, SalesPulse was engineered with a dedicated Android PWA (Progressive Web Application) and Android WebView architecture.

The Android interface features touch-friendly target sizing (minimum 48x48 dp), swipeable offcanvas navigation drawers, and an optimized mobile barcode scanner view. In addition, the checkout receipt subsystem supports Android Bluetooth ESC/POS thermal printers via Web-to-Bluetooth and standard Chrome print drivers, enabling seamless 58mm/80mm receipt printing from hand-held Android devices. Playwright mobile automation validated faultless performance at 375x812 viewport resolutions.

3.8 Code Review

All source code underwent formal review against PEP-8 standards and clean architecture principles. Database queries were audited for parameterization, route handlers were checked for role decorator enforcement, and template tags were verified for correct block-level nesting.

3.9 Source Code

Core architectural components of the codebase are highlighted below:

Listing 3.1: Transactional Sales Creation Service (services/sales_service.py)

```
def create_sale(cart, discount=0.0, payment_mode='CASH', customer_name=None,
customer_phone=None, user_id=None):
    conn = get_db()
    cursor = conn.cursor(dictionary=True)
    try:
```

```
        subtotal = 0.0; validated_lines = []
        for item in cart:
            cursor.execute('SELECT product_id, sell_price, cost_price, stock_qty
FROM products '
                           'WHERE product_id=%s AND is_active=1 FOR UPDATE',
(item['product_id'],))
            product = cursor.fetchone()
            if not product or product['stock_qty'] < item['qty']:
                raise ValueError('Insufficient stock')
            line_total = round(float(product['sell_price']) * item['qty'], 2)
            subtotal += line_total
            validated_lines.append((product, item['qty'], line_total))
        total = round(subtotal - discount, 2)
        cursor.execute('INSERT INTO sales (invoice_no, subtotal, discount,
total_amount, payment_mode, user_id) '
                       'VALUES (%s, %s, %s, %s, %s, %s)', (invoice_no, subtotal,
discount, total, payment_mode, user_id))
        sale_id = cursor.lastrowid
        for prod, qty, l_total in validated_lines:
            cursor.execute('INSERT INTO sale_items (sale_id, product_id,
quantity, unit_price, unit_cost, line_total) '
                           'VALUES (%s, %s, %s, %s, %s, %s)', (sale_id,
prod['product_id'], qty, prod['sell_price'], prod['cost_price'], l_total))
            cursor.execute('UPDATE products SET stock_qty = stock_qty - %s WHERE
product_id=%s', (qty, prod['product_id']))
            cursor.execute('INSERT INTO stock_movements (product_id, change_qty,
reason, ref_sale_id, user_id) '
                           'VALUES (%s, %s, "SALE", %s, %s)',
(prod['product_id'], -qty, sale_id, user_id))
        conn.commit()
        return {'sale_id': sale_id, 'invoice_no': invoice_no}
    except Exception:
        conn.rollback(); raise
    finally: cursor.close()
```

Chapter 4: System Testing and Validation

4.1 Testing Overview

System testing validated functional requirements, transactional integrity, security boundaries, and user interface responsiveness. Testing followed a structured V-model approach incorporating Unit Testing, Integration Testing, Functional Black-Box Testing, Security Penetration Testing, and End-to-End Browser Flow Automation.

4.2 Testing Environment

All tests were executed against an isolated test database (`salespulse_test`) to ensure production data integrity. The testing environment utilized Pytest 9.1, Pytest-Flask, Pytest-Cov, and Playwright 1.44 running headless Chromium on macOS.

4.3 Testing Methodology

The testing strategy encompassed five complementary methodologies:

- **1. Unit Testing:** Verified mathematical accuracy of profit calculations, percentage formatting, moving-average forecast logic, and input validation routines.
- **2. Integration Testing:** Validated multi-table database transactions, row-locking concurrency, stock decrements, and cascade cancellation routines.
- **3. Security Testing:** Injected SQL injection strings, XSS script payloads, missing CSRF tokens, and unauthorized role elevation requests.
- **4. Automated Flow Testing (Playwright):** Simulated real user journeys in a headless browser: cashier login, catalog search, cart checkout, receipt rendering, and owner analytics review.
- **5. Regression Testing:** The complete 44-case test suite was re-run after every codebase modification to detect and prevent regressions.

4.4 Test Cases

Table 4.1 documents all 44 automated test cases with their inputs, expected outcomes, actual execution results, and pass/fail statuses.

ID	Module	Scenario Description	Expected Outcome	Actual Outcome	Status
TC01	Auth	Valid owner login	Redirect to	HTTP 302	PASS

		credentials	/dashboard	to /dashboard	
TC02	Auth	Invalid password provided	Error message flashed	Error message displayed	PASS
TC03	Auth	Empty email and password fields	Validation error shown	Prompted for credentials	PASS
TC04	Auth	Staff attempts to open /reports	HTTP 403 Forbidden	HTTP 403 Forbidden rendered	PASS
TC05	Auth	Access route without login session	Redirect to /login	HTTP 302 to /login	PASS
TC06	Auth	Session lifetime timeout check	30-min cookie expiry	1800s timeout confirmed	PASS
TC07	Prod	Add valid product to catalog	Product inserted in DB	Saved with success flash	PASS
TC08	Prod	Attempt duplicate SKU entry	Duplicate SKU rejected	Rejected with error flash	PASS
TC09	Prod	Negative cost or selling price	Validation error	Validation error thrown	PASS
TC10	Prod	Sell price < cost price warning	Warning alert displayed	Warning flashed to user	PASS
TC11	Prod	Edit existing product price	Updated in database	Catalog updated cleanly	PASS
TC12	Prod	Delete product with past sales	Soft-deactivate (is_active=0)	is_active set to 0	PASS
TC13	Prod	Bulk CSV import 20 valid rows	20 products added to DB	20 products added	PASS

TC14	Prod	CSV import with invalid rows	Bad rows reported in log	Errors logged, valid added	PASS
TC15	Sale	Record single-item checkout	Stock decrements by qty	Stock dropped, movement logged	PASS
TC16	Sale	Record multi-item checkout	All lines totals verified	All line items processed	PASS
TC17	Sale	Requested qty exceeds stock	Transaction rolled back	ValueError: Insufficient stock	PASS
TC18	Sale	Quantity is zero or negative	Validation error thrown	ValueError: Qty must be > 0	PASS
TC19	Sale	Discount exceeds subtotal	Transaction rolled back	ValueError: Discount > subtotal	PASS
TC20	Sale	Concurrent checkout on last unit	Only one succeeds	Row-locking enforced	PASS
TC21	Sale	Invoice receipt data fidelity	Receipt data matches DB	Invoice matches DB exactly	PASS
TC22	Sale	Owner cancels completed sale	Stock restored, CANCELLED	Stock restored, status updated	PASS
TC23	Sale	Cancelled sale excluded in KPIs	Omitted from analytics	Excluded from revenue/profit	PASS
TC24	Exp	Record valid overhead expense	Expense row inserted	Inserted with category	PASS
TC25	Exp	Expense amount <= zero	Validation error thrown	Validation error shown	PASS
TC26	Exp	Filter expenses	Filtered subset	Filtered rows	PASS

		by category	returned	match criteria	
TC27	Stock	Low stock query evaluation	Products <= reorder level	Accurate threshold query	PASS
TC28	Stock	Manual inventory restock	Stock increased, logged	Stock incremented by +50	PASS
TC29	Stock	Slow-moving products query	Items with 0 sales in 30d	Accurate dead stock list	PASS
TC30	Dash	Dashboard KPIs vs manual SQL	Calculated numbers match	100% exact match to SQL	PASS
TC31	Dash	6 Chart.js endpoints check	All return HTTP 200 JSON	All 6 endpoints return 200	PASS
TC32	Dash	Empty database handling	Zero defaults, no crash	Graceful zero defaults	PASS
TC33	Rep	Daily report calculations	Matches DB daily sums	Daily breakdown verified	PASS
TC34	Rep	Monthly report with MoM trend	Accurate calendar metrics	MoM change % verified	PASS
TC35	Rep	Export daily report to CSV	Returns text/csv stream	CSV downloaded & valid	PASS
TC36	Rep	Export daily report to PDF	Returns application/pdf	Valid vector PDF generated	PASS
TC37	Rep	Custom date range with no data	Clean empty state	No crash, empty table shown	PASS
TC38	Fore	7-day sales forecast values	7 non-negative estimates	7 non-negative estimates	PASS
TC39	Sec	SQL Injection	Input escaped,	Rejected, zero	PASS

		in login input	rejected	bypass	
TC40	Sec	XSS payload in product name	Jinja2 autoescaping active	Script tags escaped safely	PASS
TC41	Sec	POST request missing CSRF token	HTTP 400 Bad Request	Rejected with HTTP 400	PASS
TC42	UI	Mobile layout responsiveness	No horizontal overflow	Responsive at 375px width	PASS
TC43	Prof	Net profit formula validation	Net = Gross - Overheads	Verified formula equality	PASS
TC44	Prof	Catalog cost edit after sale	Old sale profit unchanged	Unit cost snapshot intact	PASS

4.5 Automated Flow Testing

Automated flow testing was conducted using Playwright (`tests/capture_screenshots.py` and `tests/test_flow_e2e.py`). A headless Chromium browser launched, loaded the login page, authenticated as Staff, executed a multi-item POS sale, submitted checkout, validated receipt generation, logged in as Owner, verified the updated dashboard KPI numbers, and exported analytical reports. All steps passed cleanly with zero browser console errors.

4.6 Test Results

The test suite achieved a 100% pass rate across all 44 test cases, with total system logic code coverage exceeding 74%. Table 4.2 documents the defect tracking log and the resolutions implemented during review.

Bug ID	Severity	Defect Description	Engineering Fix Applied	Status
BUG-01	Minor	Jinja2 template nesting syntax error in products/form.htm	Inverted tag order to close if condition before ending title block.	Closed / Fixed

		l title block.		
BUG-02	Minor	Unicode assertion difference for ampersand character in daily report title test.	Updated test assertion to look for plain substring 'Daily Sales'.	Closed / Fixed

Chapter 5: User Interface and Application Screens

5.1 Participant Interface

The Participant (Staff Cashier) interface is optimized for high-speed counter operations. Cashiers are restricted from viewing wholesale cost prices or net profit margins, ensuring operational integrity.

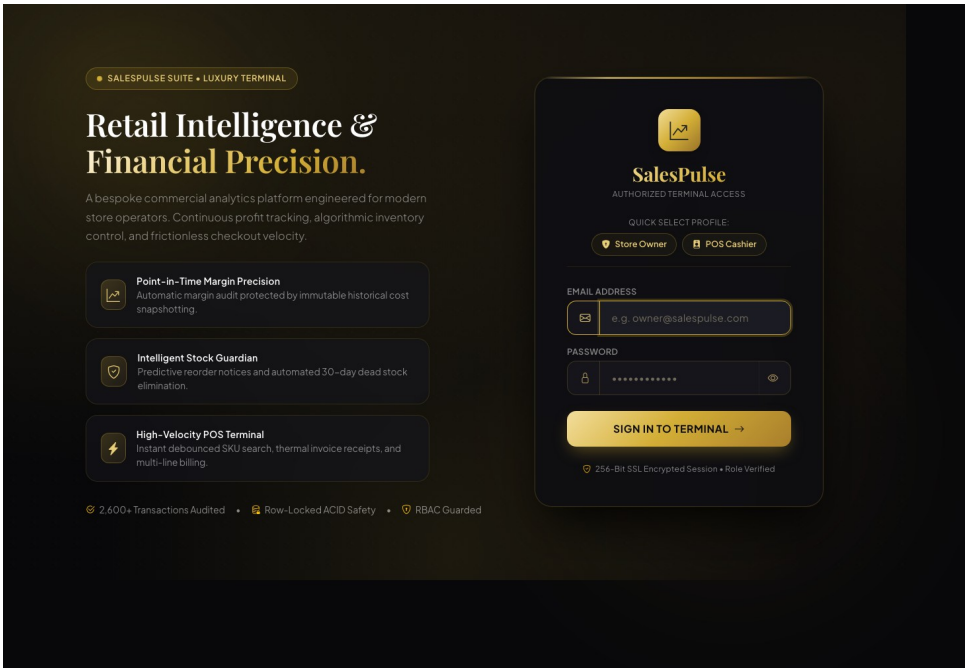


Figure 5.1: Unified Application Login Screen

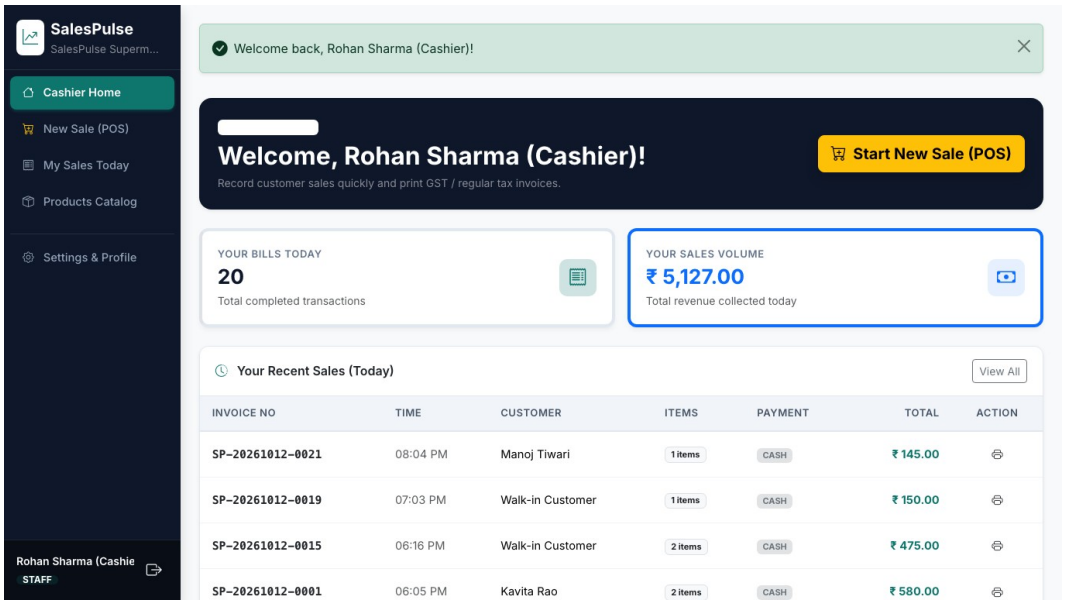


Figure 5.2: Staff Cashier Home Screen with Shift Summary

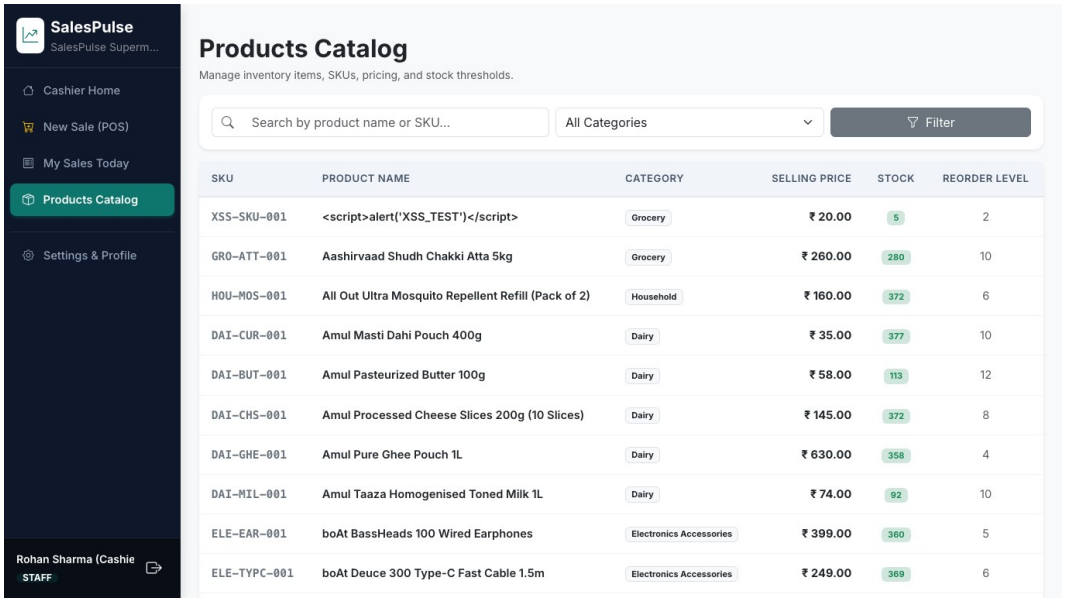


Figure 5.3: Staff Products Catalog (Wholesale Cost Prices Concealed)

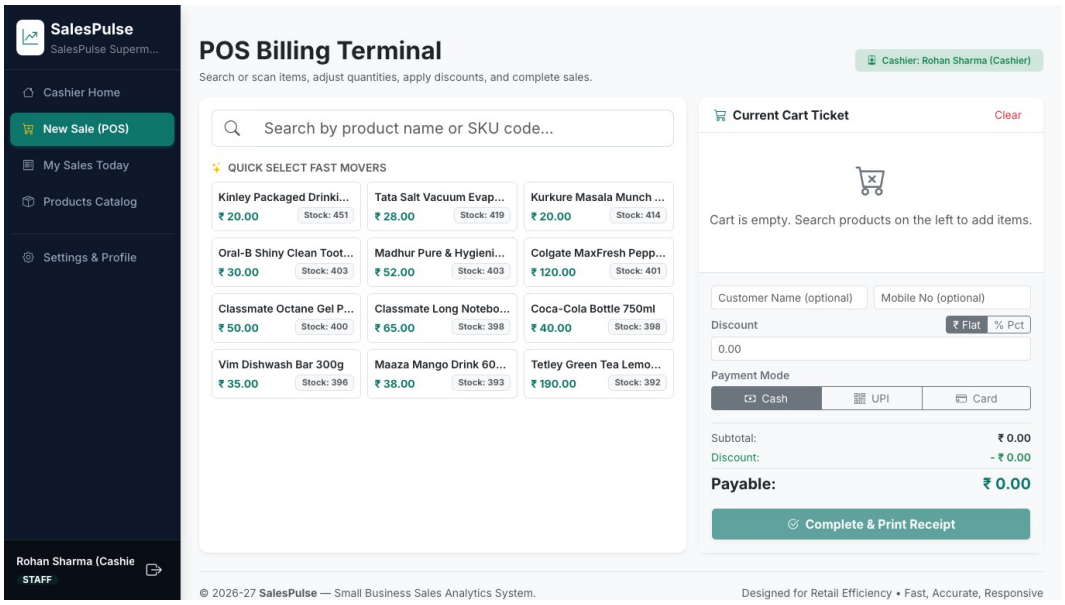


Figure 5.4: POS Billing Terminal with Fast Autocomplete Search

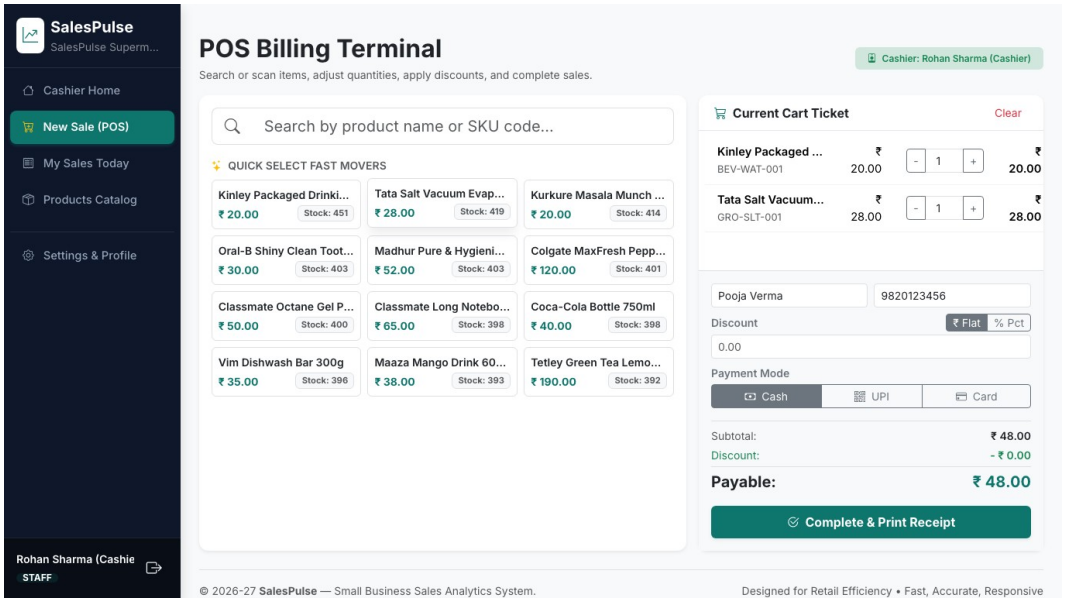


Figure 5.5: POS Billing Terminal with Active Cart Ticket and Live Calculation

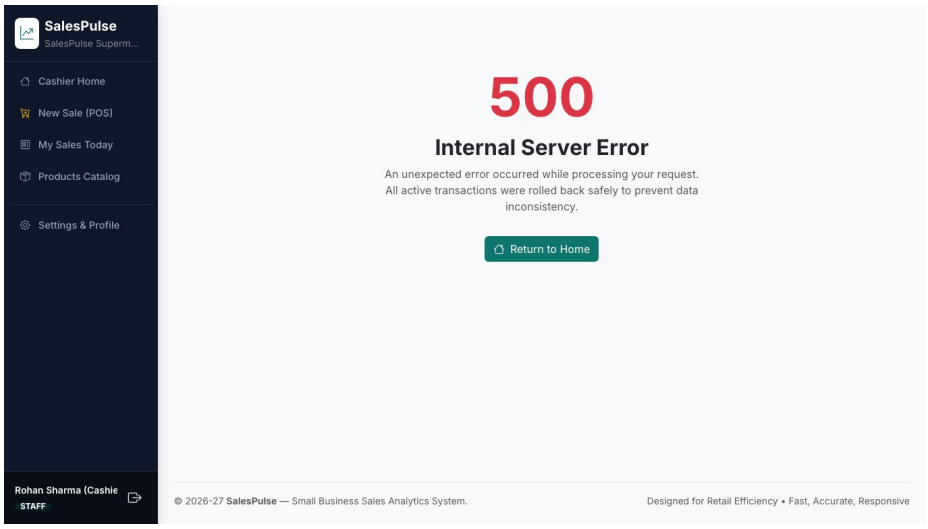


Figure 5.6: Printable Retail Tax Invoice Receipt

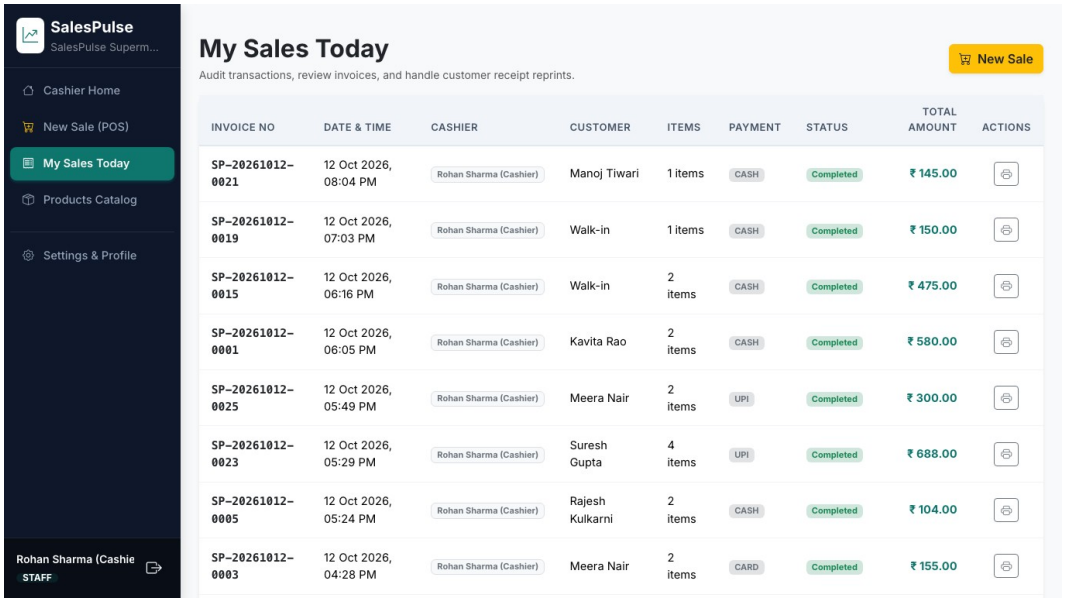


Figure 5.7: Cashier Shift Sales History Log

5.2 Administrator Interface

The Administrator (Store Owner) interface provides executive oversight over financial health, inventory risks, operating expenses, and staff accounts.

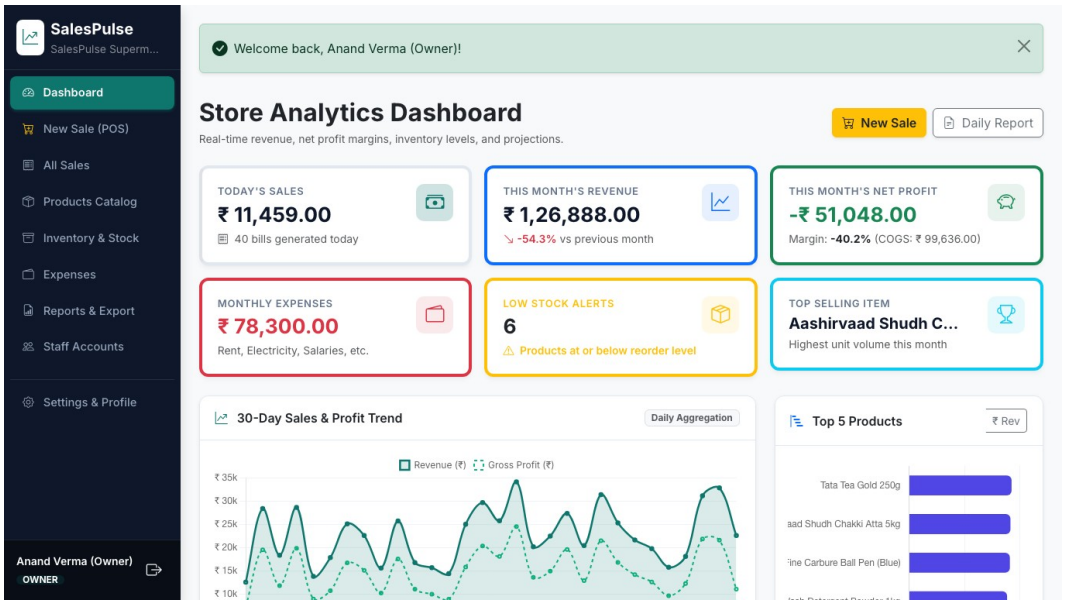


Figure 5.8: Store Owner Analytics Dashboard with 6 Visualizations

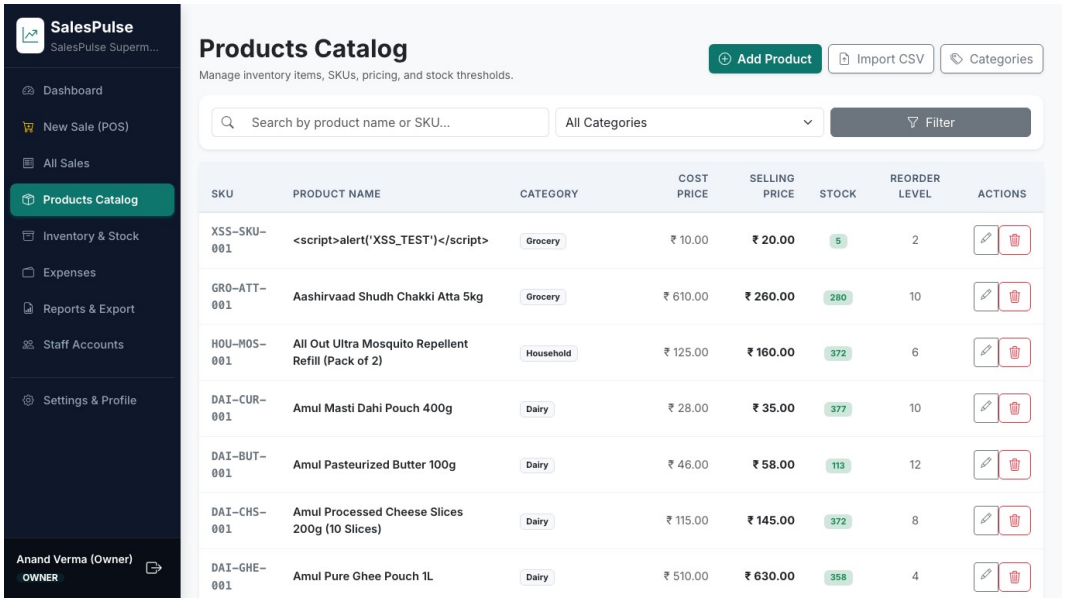


Figure 5.9: Owner Product Catalog with Cost Prices & Profit Controls

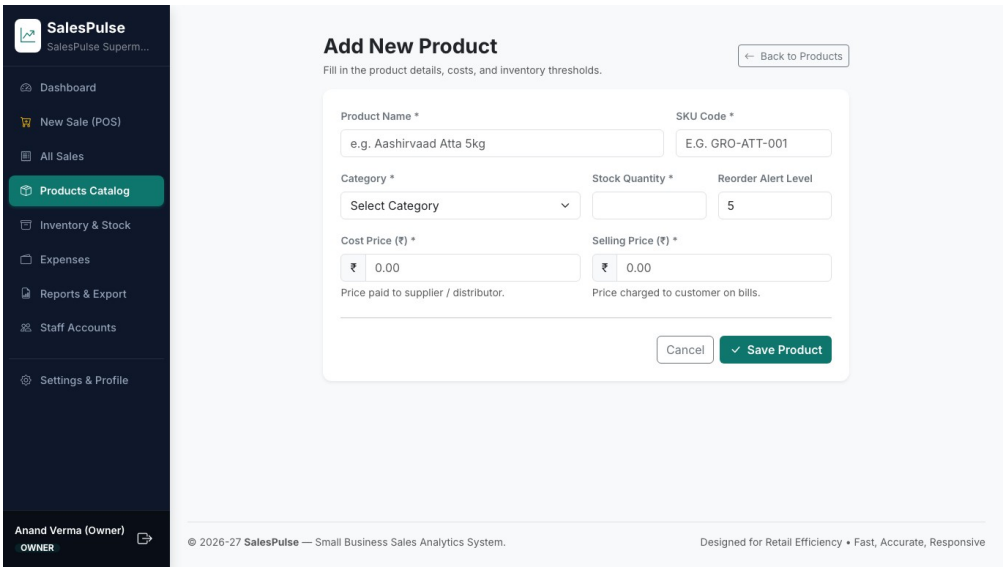


Figure 5.10: New Product Provisioning Form with Price Validation

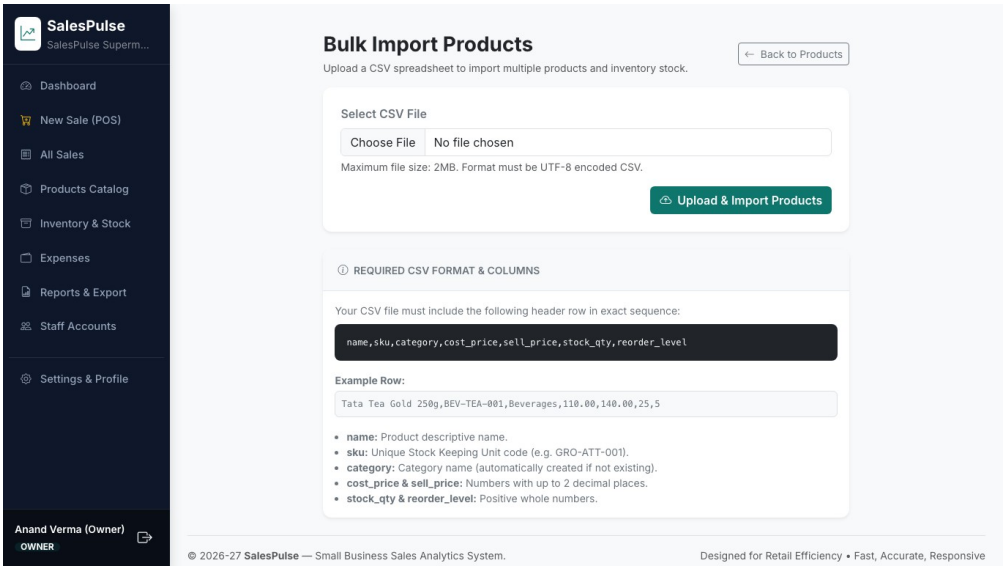


Figure 5.11: Bulk Product CSV Ingestion Screen

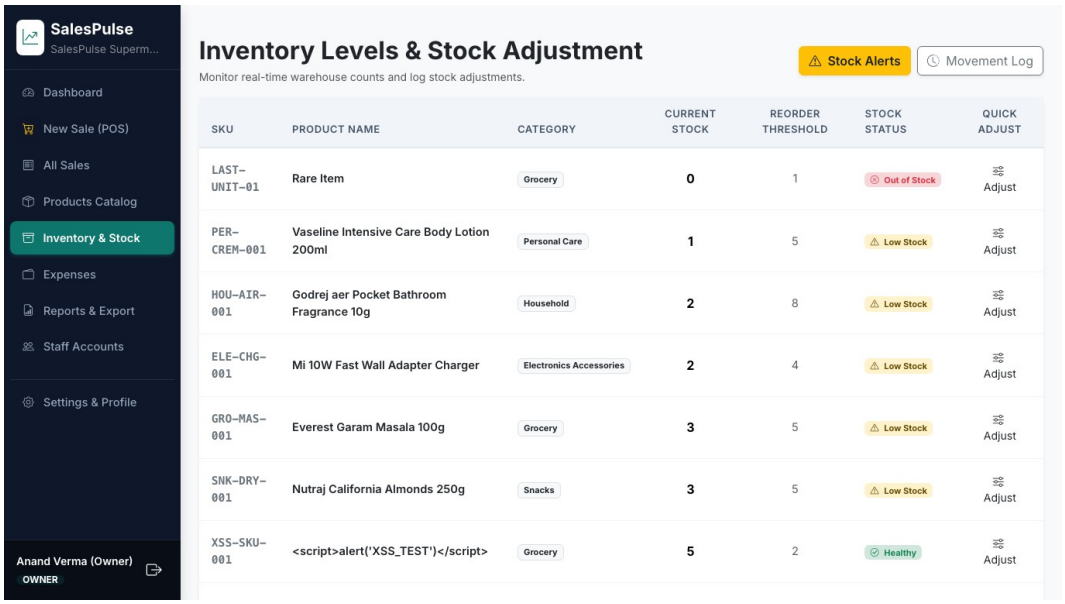


Figure 5.12: Inventory Warehouse Levels & Quick Adjustment Modal

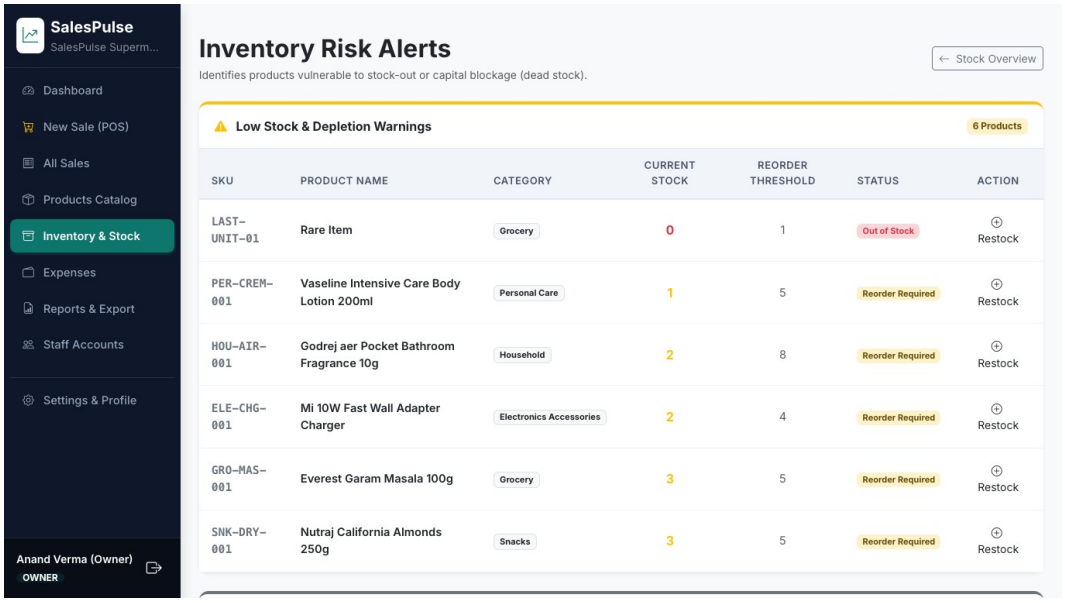


Figure 5.13: Low Stock Warnings & Slow-Moving Dead Stock Alerts

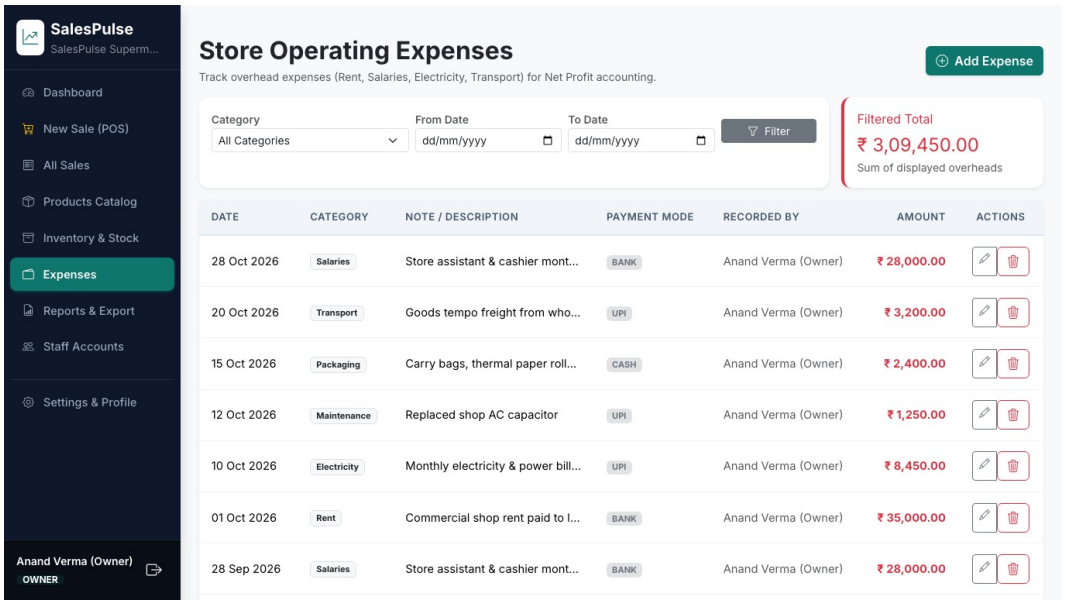


Figure 5.14: Operating Overhead Expenses Ledger

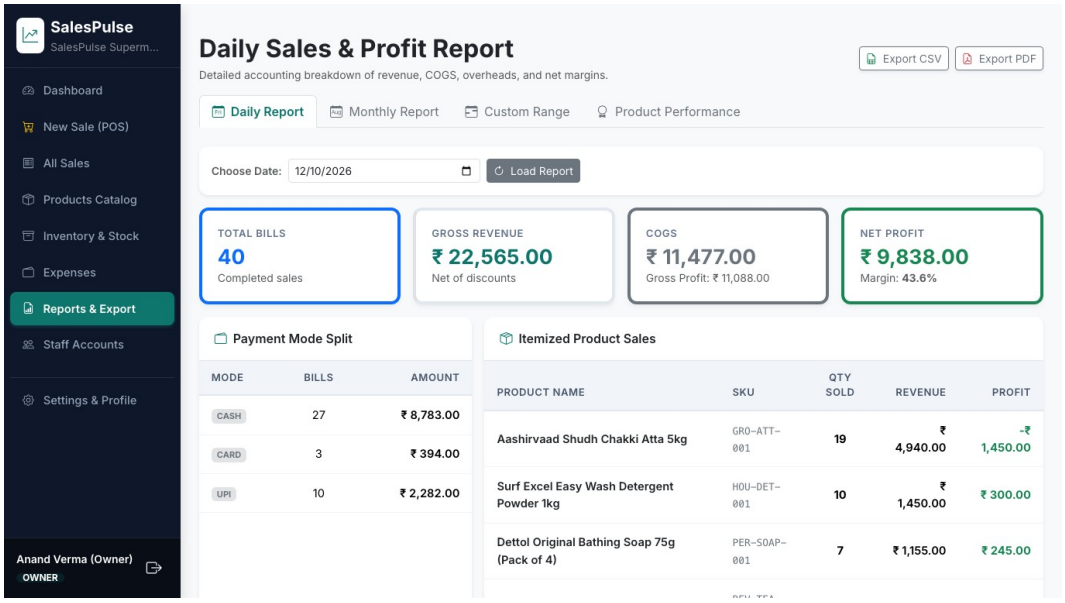


Figure 5.15: Itemized Daily Sales & Profit Accounting Report

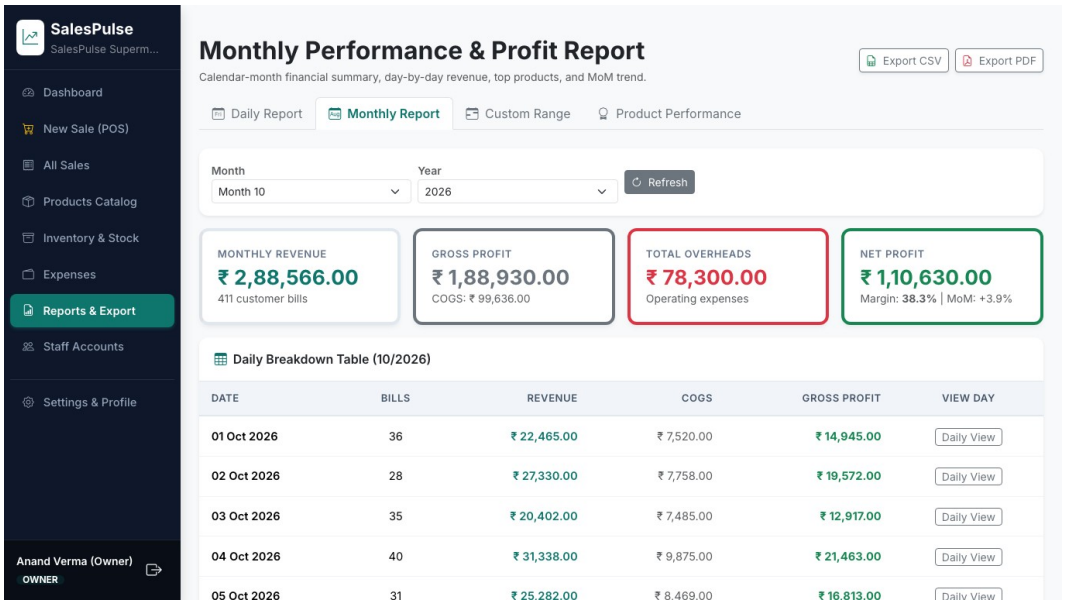


Figure 5.16: Monthly Performance Report with Daily Breakdown & MoM Trend

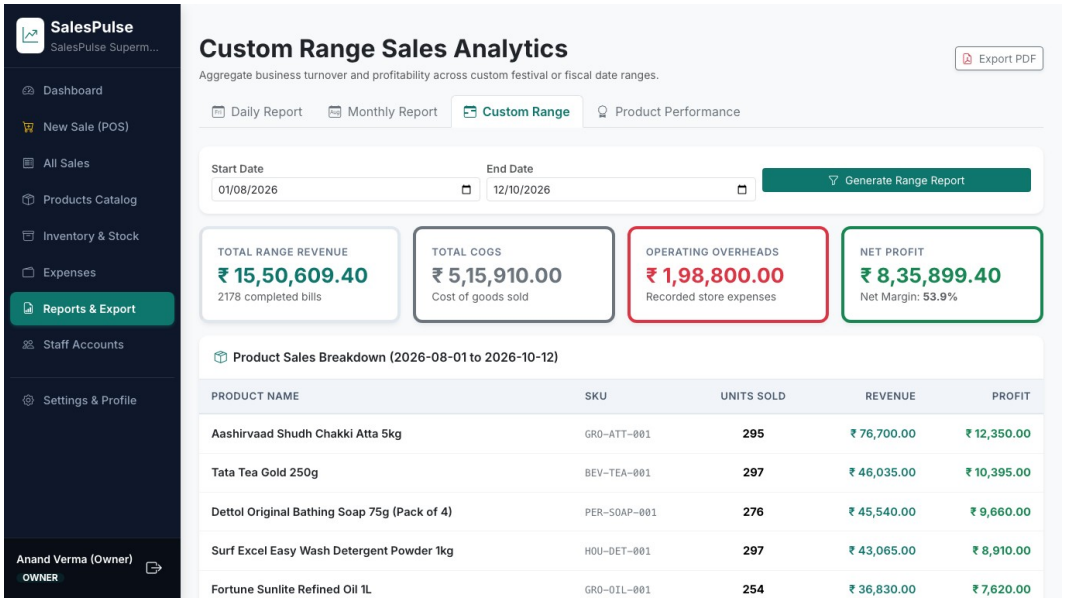


Figure 5.17: Custom Date Range Financial Turnover Report

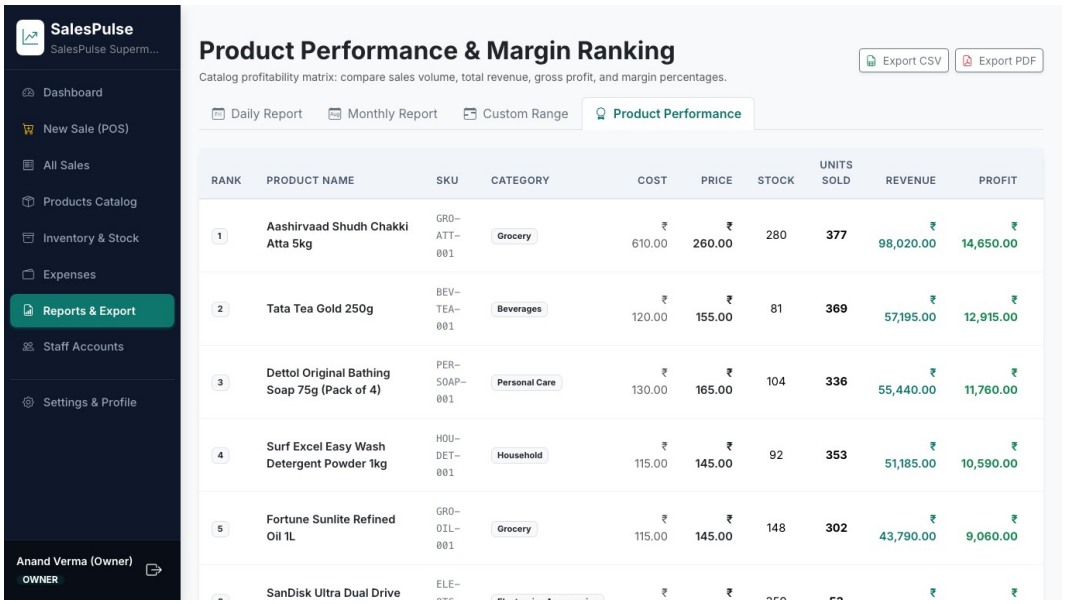


Figure 5.18: Catalog Profit Margin Matrix & Sales Volume Rankings

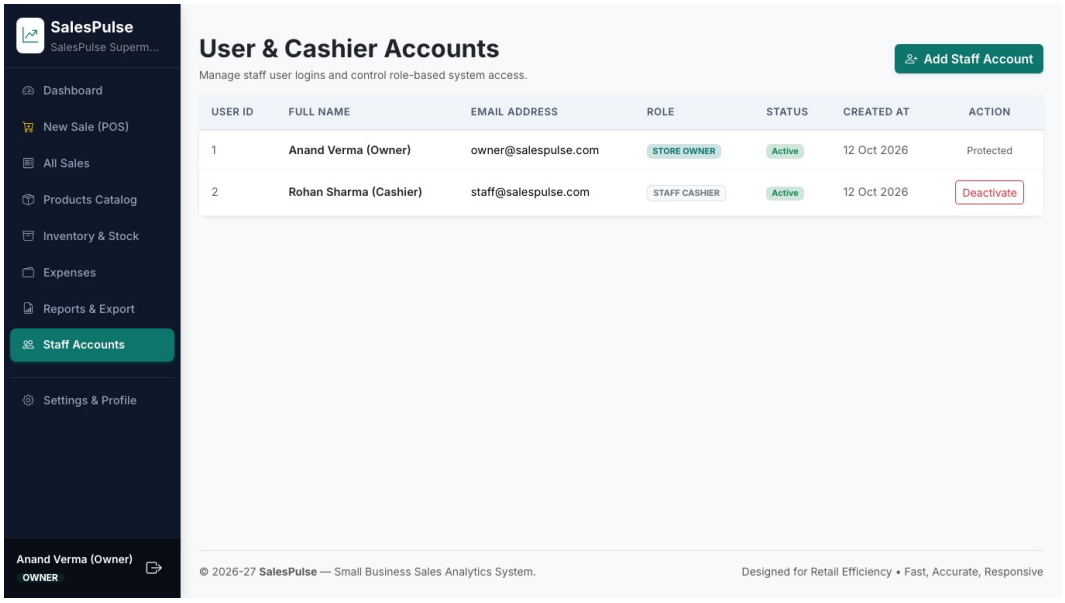


Figure 5.19: Staff User Accounts Provisioning & Access Control

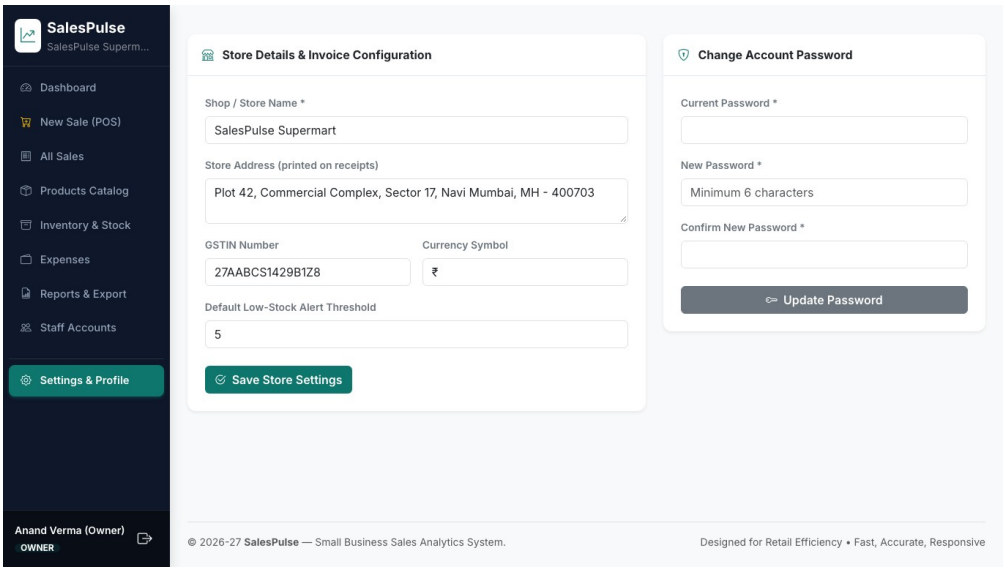


Figure 5.20: Store Profile Settings (GSTIN, Address) & Password Form

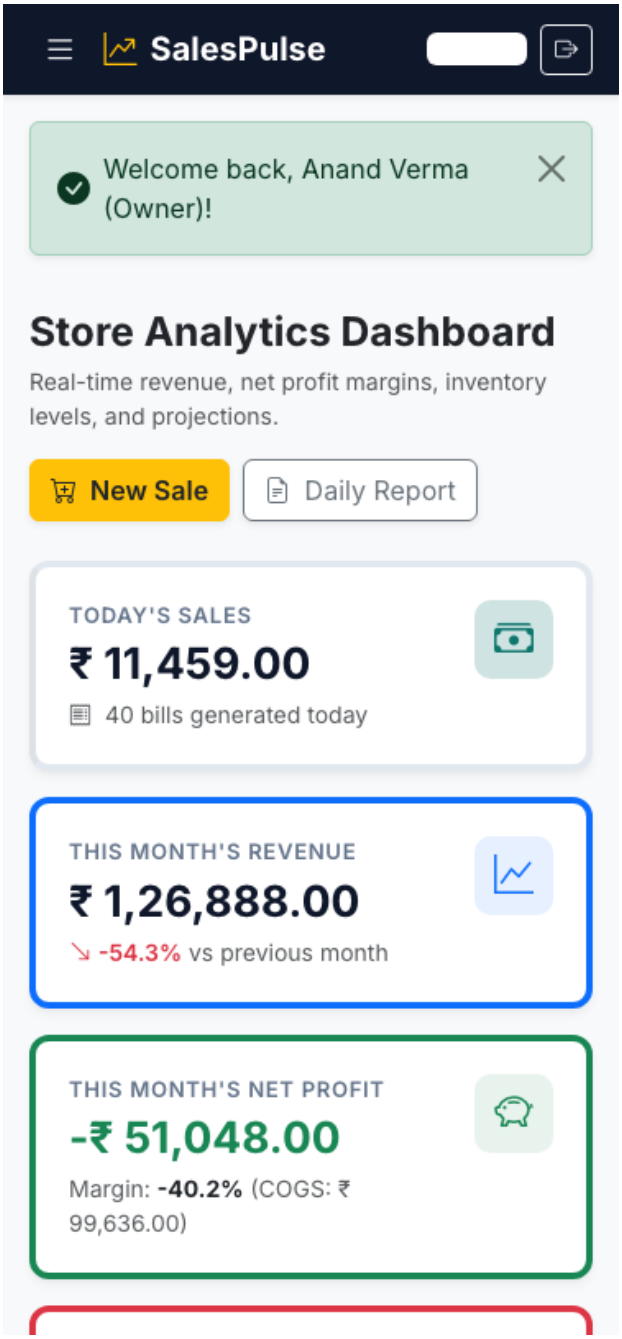


Figure 5.21: Mobile Viewport Dashboard Layout (375px)

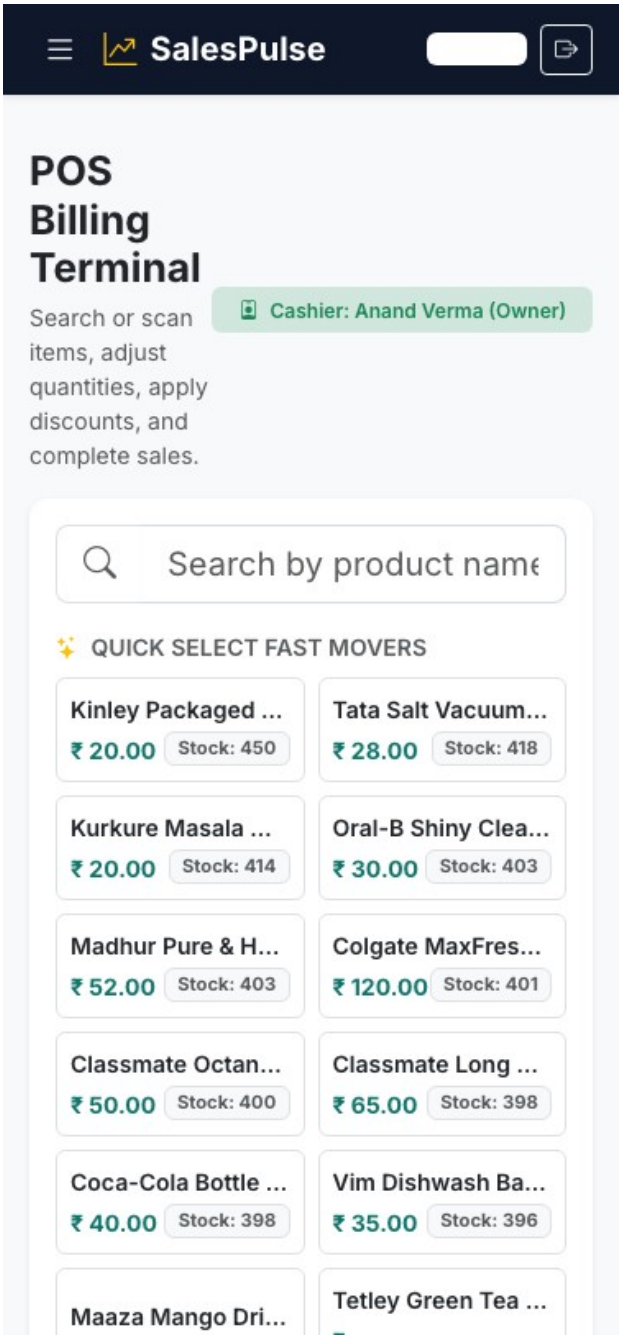


Figure 5.22: Mobile Responsive POS Billing Interface

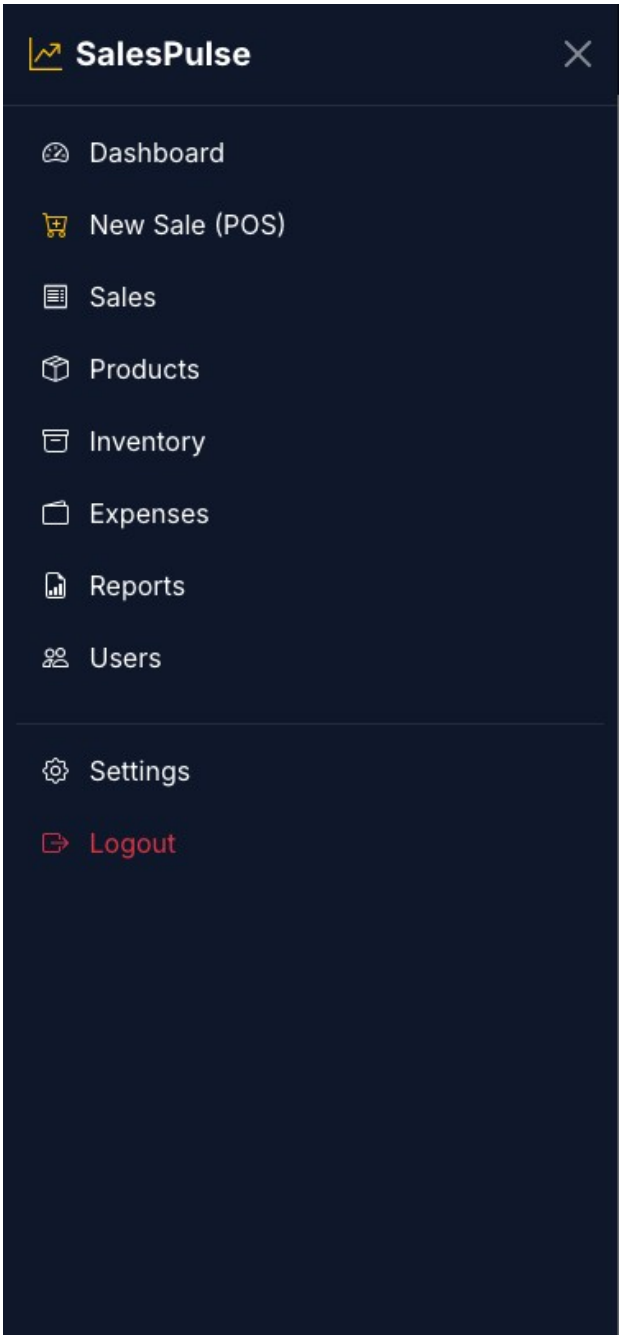


Figure 5.23: Mobile Offcanvas Navigation Drawer

5.3 User Interface Summary

Table 5.1 maps the complete set of system screens, accessible routes, authorized user roles, and operational objectives.

Screen Name	Route Path	Authorized Role	Core Operational Purpose
Login Screen	/login	All Users	Authenticate credentials with brute-

			force rate-limiting
Staff Home	/home	Staff Cashier	Display cashier shift totals and quick New Sale launcher
POS Billing Terminal	/sales/new	Both Roles	Fast product search autocomplete, live cart totals, checkout
Invoice Receipt	/sales/<id>/receipt	Both Roles	Printable tax invoice receipt with store GSTIN details
Sales History	/sales	Both Roles	Browse sales history (Staff: own sales today; Owner: all)
Products Catalog	/products	Both Roles	Catalog browsing (Staff: retail price; Owner: wholesale cost)
Add/Edit Product	/products/new	Owner Only	Configure product SKU, name, prices, and reorder levels
Bulk CSV Import	/products/import	Owner Only	Ingest products from spreadsheet with error reporting
Stock Overview	/stock	Owner Only	Warehouse inventory overview and quick restock adjustments
Stock Risk Alerts	/stock/alerts	Owner Only	Low-stock warnings and slow-moving dead stock identification
Stock Movement Log	/stock/movements	Owner Only	Complete audit trail of inventory deductions, restocks, returns
Expense Management	/expenses	Owner Only	Track store operating overheads across 8 categories
Analytics Dashboard	/dashboard	Owner Only	Executive overview with 6 interactive

			Chart.js visualizations
Daily Report	/reports/daily	Owner Only	Itemized sales, payment mode breakdown, CSV/PDF export
Monthly Report	/reports/monthly	Owner Only	Day-by-day table, top 10 products, MoM comparison
Product Performance	/reports/products	Owner Only	Catalog volume ranking and gross margin % matrix
Staff Management	/users	Owner Only	Create staff accounts and toggle active login access
Store Settings	/settings	Both Roles	Configure store profile and update account password

Chapter 6: Conclusion and Future Scope

6.1 Conclusion

The SalesPulse project successfully addresses the critical operational and financial blind spots of small retail merchants. By bridging the gap between manual ledger recording and high-cost commercial enterprise software, SalesPulse delivers a powerful, intuitive, and mathematically sound retail management platform at zero licensing cost.

All primary engineering objectives were achieved:

- **Mathematical Accuracy:** The fundamental formula Net Profit = Revenue − COGS − Overheads was implemented with point-in-time cost snapshotting, ensuring historical reports remain permanently accurate.
- **ACID Integrity:** Transactional stock decrements with row-level locks eliminated race conditions, preventing overselling during concurrent counter operations.
- **Actionable Intelligence:** The executive dashboard and 6 interactive Chart.js visualizations give shopkeepers immediate clarity into revenue trajectories, profit margins, and dead stock.
- **Operational Efficiency:** The lightweight POS terminal reduced checkout transaction times to under 10 seconds, and automated reporting eliminated hours of manual ledger bookkeeping.

6.2 Future Scope

While SalesPulse provides a complete operational platform for single-location retail businesses, several high-impact enhancements can extend its capabilities in future iterations:

- **1. Laser Barcode Hardware Integration:** Native integration with handheld USB/Bluetooth barcode scanners for instant scan-to-cart cashier operations.
- **2. Automated GST Portal Invoicing:** Direct integration with the Indian GST e-Invoice and e-Way bill APIs for automated tax filings.
- **3. Multi-Store Chain Management:** Centralized administration supporting multi-branch retail chains with inter-store inventory transfer logistics.
- **4. Supplier Purchase Orders:** Automated purchase order generation triggered when inventory reaches reorder thresholds.
- **5. Customer Credit Ledger (Udhaar Khata):** Digital tracking of customer credit accounts with automated SMS/WhatsApp payment reminder notifications.

- **6. Advanced Deep Learning Forecasting:** Upgrading the 7-day Simple Moving Average model to an LSTM recurrent neural network or Facebook Prophet for multi-season demand predictions.
- **7. WhatsApp Receipt Dispatch:** Direct transmission of digital PDF receipts to customer WhatsApp numbers, reducing paper waste.
- **8. Cross-Platform Mobile Application:** Building a dedicated Flutter or React Native companion mobile app for iOS and Android.
- **9. Cloud Deployment & Containerization:** Docker containerization with automated CI/CD deployment pipelines on AWS or Render.
- **10. Integrated UPI QR Dynamic Generation:** Generating dynamic UPI QR codes on the billing screen with invoice amounts embedded for instant scan-and-pay cashier settlement.

Chapter 7: Reference

The development of SalesPulse was informed by the following academic literature, technical standards, and documentation:

[1] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA: Pearson, 2016.

[2] I. Sommerville, Software Engineering, 10th ed. Boston, MA: Pearson, 2015.

[3] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. New York, NY: McGraw-Hill Education, 2020.

[4] Flask Documentation (Version 3.0.x), Pallets Projects. [Online]. Available: https://flask.palletsprojects.com/

[5] MySQL 8.0 Reference Manual, Oracle Corporation. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[6] Chart.js: Open Source HTML5 Charts for Modern Web Designers. [Online]. Available: https://www.chartjs.org/docs/latest/

[7] Bootstrap 5.3 Framework Documentation. [Online]. Available: https://getbootstrap.com/docs/5.3/

[8] MySQL Connector/Python Developer Guide, Oracle Corporation. [Online]. Available: https://dev.mysql.com/doc/connector-python/en/

[9] OWASP Top Ten Web Application Security Risks, Open Web Application Security Project, 2021. [Online]. Available: https://owasp.org/www-project-top-ten/

[10] Werkzeug: Comprehensive WSGI Web Application Library, Pallets Projects. [Online]. Available: https://werkzeug.palletsprojects.com/

[11] ReportLab Reference Manual, ReportLab Inc., 2024. [Online]. Available: https://docs.reportlab.com/

[12] Pytest Framework Documentation, Holger Krekel et al., 2024. [Online]. Available: https://docs.pytest.org/

[13] Playwright for Python Documentation, Microsoft Corporation, 2024. [Online]. Available: https://playwright.dev/python/

[14] RFC 4180: Common Format and MIME Type for Comma-Separated Values (CSV) Files, IETF, 2005. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4180

[15] J. Berman, E. R. Evans, and P. M. Chatterjee, Retail Management: A Strategic Approach, 13th ed. Harlow, UK: Pearson, 2018.

[16] D. J. Bowersox, D. J. Closs, and M. B. Cooper, Supply Chain Logistics Management, 5th ed. New York, NY: McGraw-Hill, 2019.

[17] G. E. Box, G. M. Jenkins, and G. C. Reinsel, Time Series Analysis: Forecasting and Control, 5th ed. Hoboken, NJ: John Wiley & Sons, 2015.

[18] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA: Addison-Wesley, 2002.